ТЕХНОЛОГИИ
РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

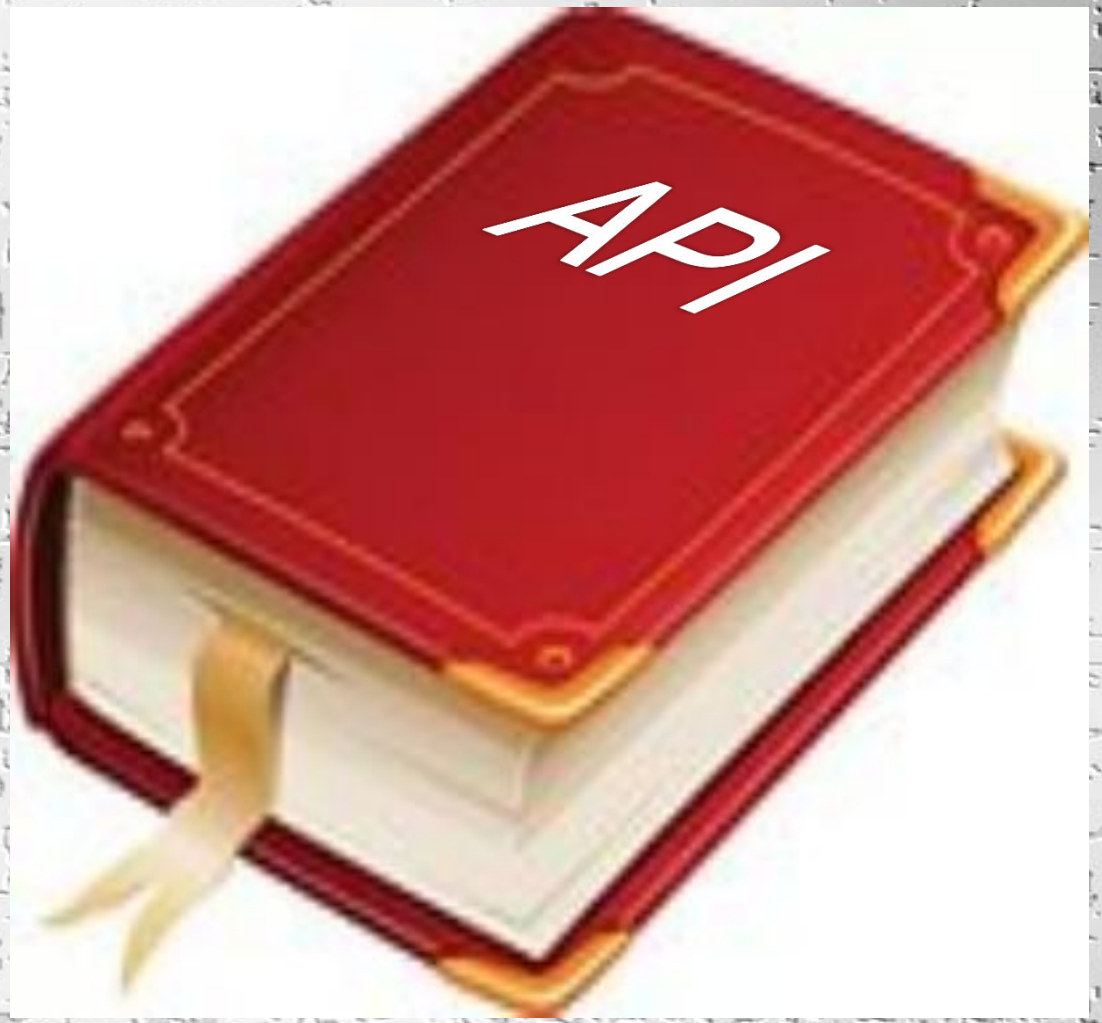
ТРПО

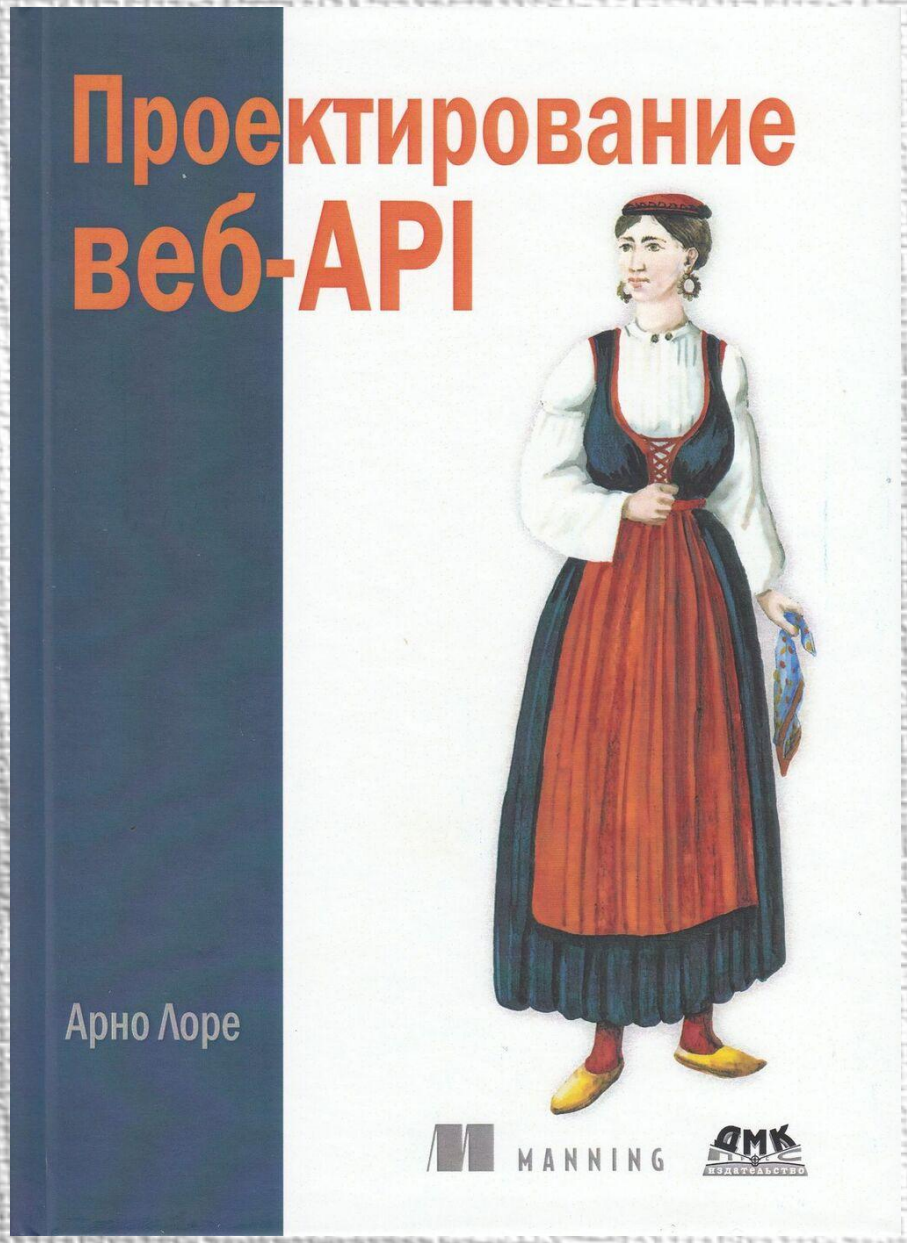
(Программная инженерия)

Часть 5. Объектно-ориентированный анализ и проектирование. Проектирование API

© Ржавин В.В.

КНИГИ





Арно Лоре

«Проектирование веб-API»

API позволяет разработчикам выполнять интеграцию с приложением без знания подробностей на уровне кода. Независимо от того, используете ли вы установленные стандарты, такие как REST и OpenAPI, или более новые подходы, такие как GraphQL или gRPC, освоение проектирования API – своего рода суперспособность. Благодаря этому пользоваться вашими веб-сервисами станет легче, и ваши клиенты, как внутренние, так и внешние, это оценят.

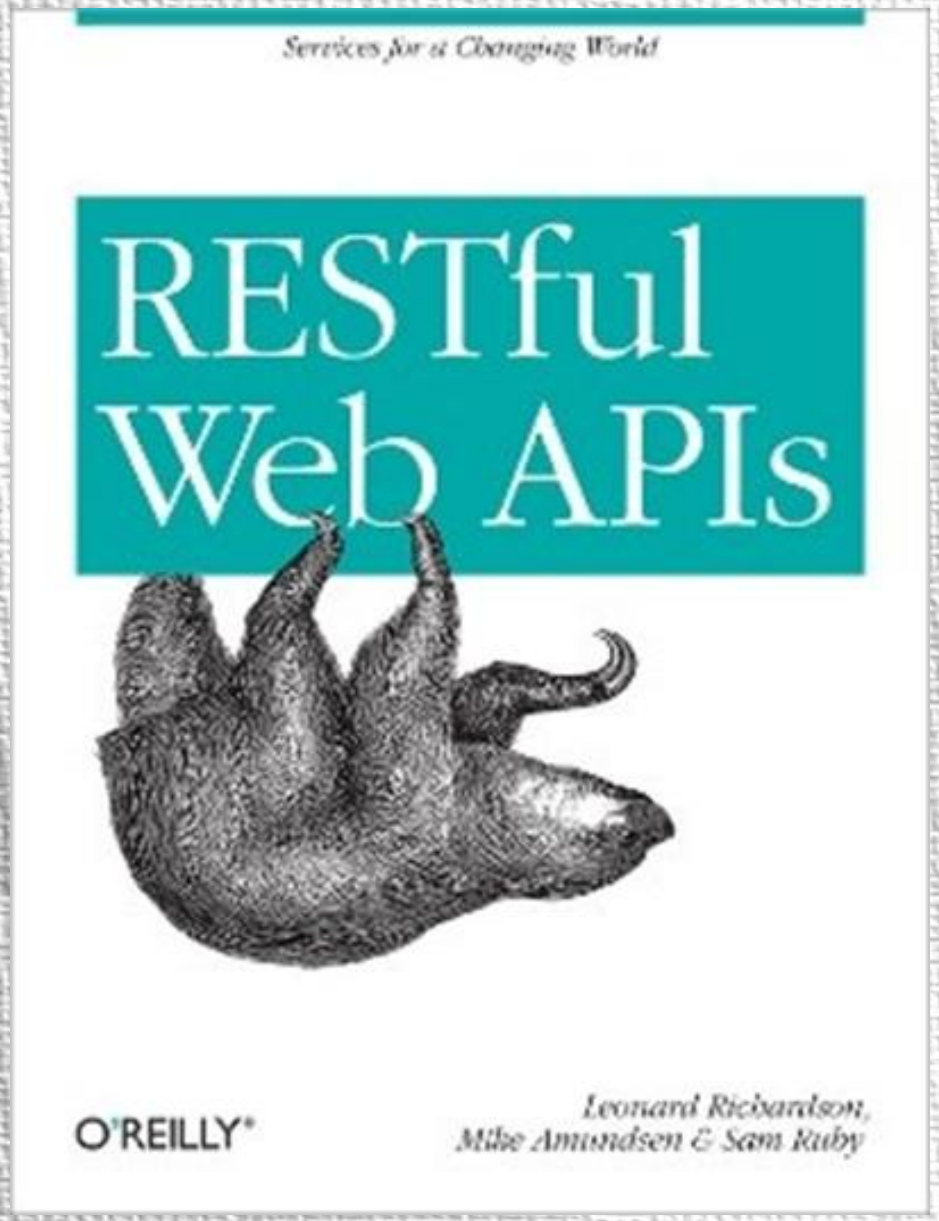


API

Сергей Константинов

Эта книга посвящена проектированию API: как правильно выстроить архитектуру, начиная с высокоуровневого планирования и заканчивая деталями реализации конкретных интерфейсов.

В книге рассматриваются следующие темы:



RESTful Web APIs (2013)

Leonard Richardson, Mike Amundsen, Sam Ruby

С помощью этого практического руководства вы узнаете, что нужно для разработки REST API приложений. Вы изучите концепции REST, изучите различные стратегии создания API-интерфейсов и познакомитесь с основными принципами данного типа программирования.

В книге рассматриваются следующие темы:

- ✓ создание простого API;
- ✓ DDD;
- ✓ паттерн проектирования Коллекция;
- ✓ работа с протоколом HTTP и многое другое.

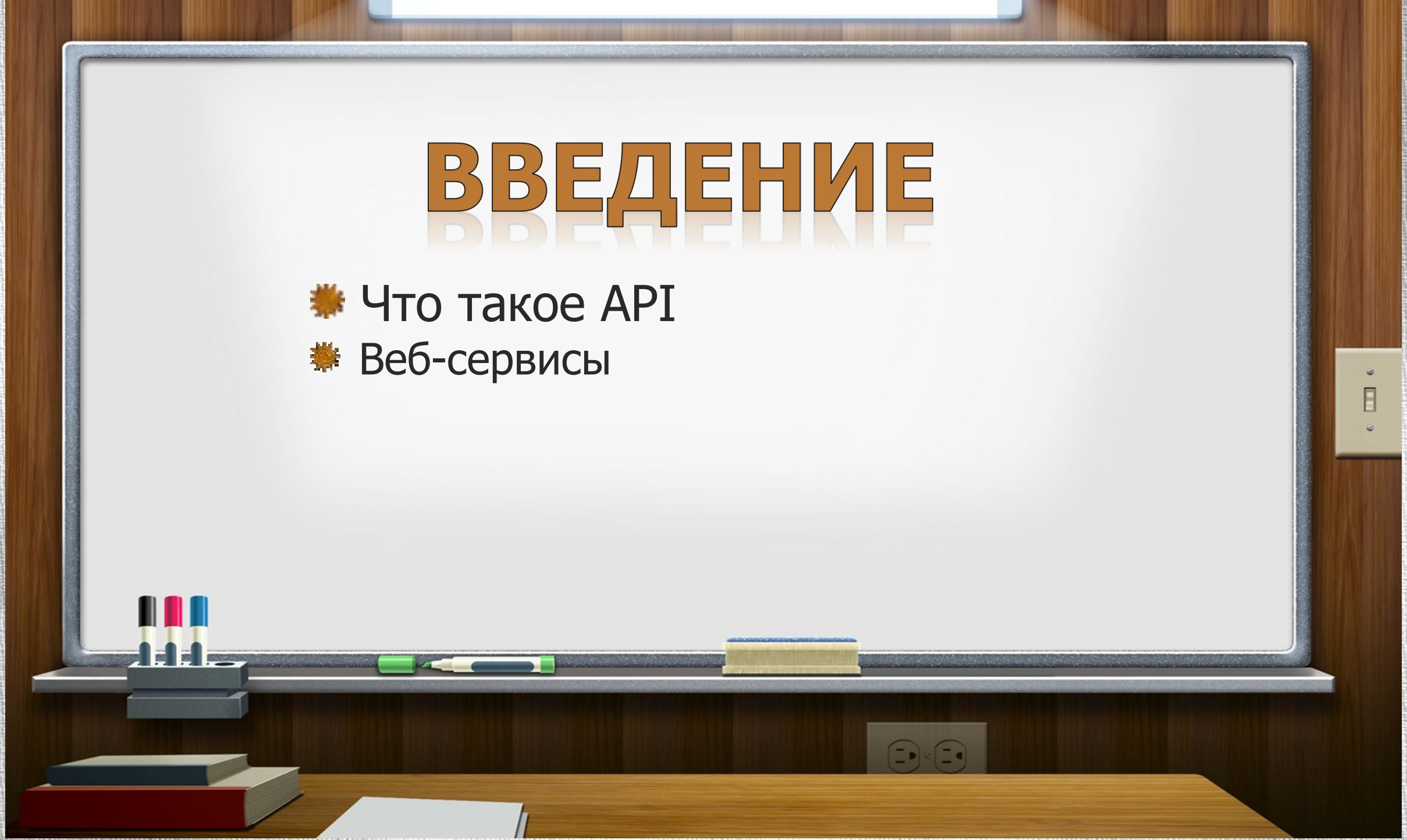

лекции

ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ИНТЕРФЕЙСА (API)

- Введение
- REST API
- Проектирование REST API
- Реализация API

ВВЕДЕНИЕ

- Что такое API
- Веб-сервисы



Что такое API

Что такое API

В общем, API (Application Programming Interface) обеспечивает взаимодействие между двумя системами. Это как винтик, связывающий две детали, или как шестеренка, при помощи которой приводятся в движение две соседние шестеренки.



API — это обязательство. Формальное обязательство связывать между собой различные программируемые контексты



Древнеримский акведук
(из книги «API C. Константинов»)

Акведук связывает между собой две области. Обратная совместимость нарушена ноль раз за последние две тысячи лет.

Отличие древнеримского акведука от хорошего API состоит лишь в том, что API предлагает программный контракт. Для связывания двух областей необходимо написать некоторый код.

Акведук также хорошо иллюстрирует другую проблему разработки API: вашими пользователями являются инженеры. Вы не поставляете воду напрямую потребителю: к вашей инженерной мысли подключаются заказчики путём пристройки к ней каких-то своих инженерных конструкций. Тем самым с одной стороны, вы можете обеспечить водой гораздо больше людей, нежели если бы вы сами подводили трубы к каждому крану. С другой — качество инженерных решений заказчика вы не можете контролировать, и проблемы с водой, вызванные некомпетентностью подрядчика, неизбежно будут валить на вас.

При нажатии на кнопку в пользовательском интерфейсе, запускаются внутренние функции, чтобы передать и получить информацию. Веб-API работают аналогичным образом. Но вместо того, чтобы получать информацию из одной и той же системы, веб-API вызывают удаленные сервисы в сети, чтобы получить их информацию.

В конечном счете, разработчики вызывают API незримо для пользователя для извлечения информации в свои приложения.

Что такое API?

Есть хороший пример идеального API не из мира IT и это ресторан. Если вы идете в ресторан как гость, то вам запрещено проходить на кухню. В этом случае вы должны знать, что вы можете заказать. Для этого у вас есть меню. После изучения меню вы делаете заказ официанту, который передает заказ на кухню, и он же принесет вам то, что вы заказали. Официант может принести только то, что может дать ему кухня. Как же это относится к API? Официант это API ресторана. Вы пользователь API. Меню — это документация, которая объясняет, что вы можете запросить у API. Блюдо в вашем заказе — это запрос. Кухня — это сервер, которая знает, как готовить то или иное блюдо.

Типы API

Существует несколько типов API в зависимости от того, каким образом они предоставляют доступ к функциональности и данным.

1. **Веб-API** (Web APIs). Web-API предоставляют доступ к функциональности и данным через интернет с использованием стандартных протоколов, таких как HTTP. RESTful API и SOAP API — это примеры веб-API, которые позволяют взаимодействовать с удаленными веб-сервисами.
2. **Библиотеки и SDK** (Software Development Kit). Эти API предоставляются в виде библиотек или наборов инструментов, которые разработчики могут внедрить непосредственно в свои приложения. Они часто используются для работы с конкретными языками программирования или платформами.

3. **API операционных систем**. Эти API предоставляют доступ к функциям операционных систем, таким как файловая система, сеть и управление устройствами. Примеры вклю-чают WinAPI для Windows и POSIX API для Unix-подобных операционных систем.
4. **API DB/DWH**. API, предназначенные для взаимодействия с базами данных. Примеры включают JDBC для Java и SQLAlchemy для Python.
5. **Графические API**. API для создания графических интерфейсов и визуализации данных. Примеры включают DirectX и OpenGL для 3D-графики.
6. **API социальных сетей**. Популярные социальные сети, такие как Facebook, Twitter и Instagram, предоставляют API для доступа к данным и функциям своих платформ.

7. **Файловые API**. API для работы с файлами и хранилищами. Примеры включают API для облачных хранилищ, такие как Amazon S3 и Google Cloud Storage.
8. **Микросервисные API**. Используются в архитектуре микросервисов для взаимодей-ствия между различными микросервисами.
9. **Мобильные API**. API, предназначенные для мобильных платформ, такие как Android API и iOS API.

Чаще всего когда мы говорим об API, то традиционно подразумеваем микросервисные API, а особенно протоколы REST и SOAP.

Критерии качества API

Как же дизайн API может помочь разработчику? Очень просто: API должен решать задачи максимально удобно и понятно. Путь разработчика от формулирования своей задачи до написания работающего кода должен быть максимально коротким. Это, в том числе, означает, что:

- из структуры вашего API должно быть максимально очевидно, как решить ту или иную задачу;
  - разработчику должно быть достаточно одного взгляда на документацию, чтобы понять, с помощью каких сущностей следует решать поставленную задачу;
- API должен быть читаемым:
  - в идеале, разработчик, просто глядя в номенклатуру методов, сразу пишет правильный код, не углубляясь в детали (особенно — детали реализации!);
  - немаловажно уточнить, что из интерфейсов объектов должно быть понятно не только решение задачи, но и возможные ошибки и исключения;

- API должен быть консистентен:
  - при разработке новой функциональности, т.е. при обращении к каким-то незнакомым сущностям в API, разработчик может действовать по аналогии с уже известными ему концепциями API, и его код будет работать;
  - желательно при этом, чтобы API соответствовал принципам и правилам платформы и используемого Фреймворка.

Однако статическое удобство и понятность API — это относительно простая часть.

Проблемы возникают, когда мы начинаем API развивать. Добавление новой функциональности рано или поздно приводит к тому, что некогда простой и понятный API становится наслоением разных концепций, а попытки сохранить обратную совместимость приводят к нелогичным, неочевидным и попросту плохим решениям. Отчасти это связано так же и с тем, что невозможно обладать полным знанием о будущем: ваше понимание о «правильном» API тоже будет меняться со временем.

API-first подход

В техническом смысле мы имеем в виду следующее: контракт, т.е. обязательство связывать программные контексты, предшествует реализации и определяет её. Конкретнее, речь идёт о двух принципах:

1. контракт разрабатывается и фиксируется в виде спецификации до того, как функциональность непосредственно реализована;
2. если обнаруживается несоответствие контракта и его имплементации, изменения вносятся в имплементацию, а не в контракт.

Здесь под спецификацией мы понимаем формальное машиночитаемое описание контракта на одном из языков определения интерфейсов (Interface Definition Language, IDL) — например, в виде Swagger/OpenAPI спецификации или .proto-файла.

Типы веб-API

1. REST (передача репрезентативного состояния):

- Использует стандартные методы HTTP.
- Архитектура без сохранения состояния.
- Ресурсы, идентифицируемые по URL-адресам.
- Широко используется благодаря простоте и масштабируемости.

2. SOAP (Простой протокол доступа к объектам):

- Протокол обмена структурированной информацией.
- Опирается на XML.
- Поддерживает сложные операции и более высокий уровень безопасности.
- Используется в приложениях корпоративного уровня.

3. GraphQL:

- Позволяет клиентам запрашивать именно те данные, которые им нужны.
- Уменьшает избыточную и недостаточную выборку данных.
- Поддерживает более гибкие запросы по сравнению с REST.

4. gRPC:

- Использует HTTP/2 для транспорта и буферы протоколов для сериализации данных.
- Поддерживает двунаправленную потоковую передачу.
- Высокая производительность и подходит для микросервисов.

Что представляет вызов API с технической точки зрения

1. Как пользователь приложения, которому необходимо выполнить задачу, вы инициируете задачу из своего приложения, создавая запрос.
2. API совершает вызов к веб-серверу, передавая запрос. API знает, куда отправлять запрос, поскольку он передается в конечную точку API, обычно — URL сервера.
3. Запрос выполняет стороннее приложение или база данных, предоставляющие такой сервис.

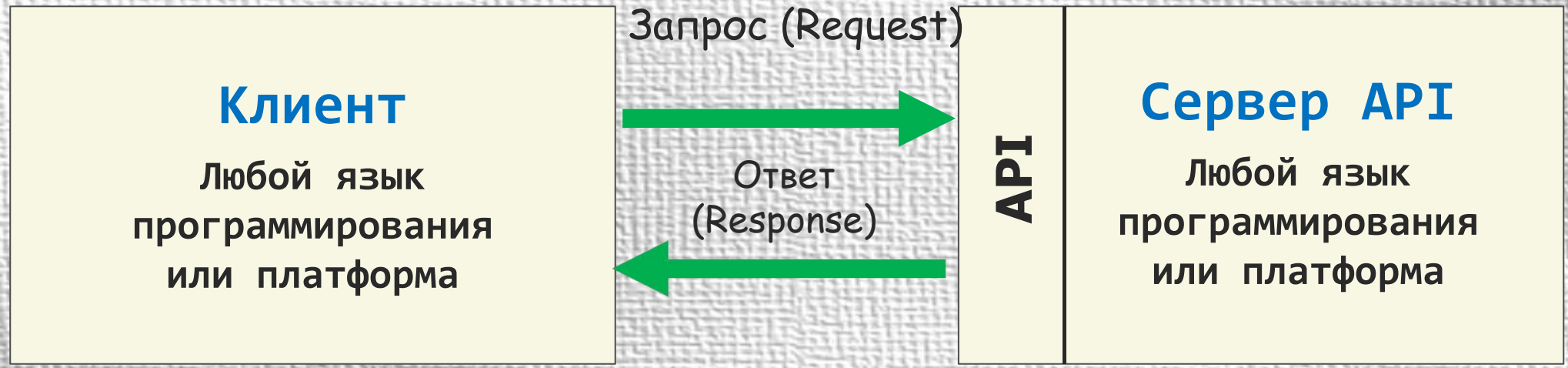
Веб-сервисы

API, использующие протокол HTTP = веб-сервисы

«Веб-сервис» - это веб-приложение, предоставляющее ресурсы в формате, используемом другими компьютерами. Веб-сервисы включают в себя различные типы API, в том числе REST и SOAP API. Веб-сервисы - это, в основном, запросы и ответы между клиентами и серверами (компьютер запрашивает ресурс, а веб-сервис отвечает на запрос).

API, использующие протокол HTTP для передачи запросов и ответов, рассматриваются как «веб-сервисы». В случае веб-сервисов клиент, делающий запрос на ресурс, и сервер API, предоставляющий ответ, могут использовать любой язык программирования или платформу. Не имеет значения, какой ЯП или платформа будут использоваться, потому что запрос сообщения и ответ сделаны через общий веб-протокол HTTP.

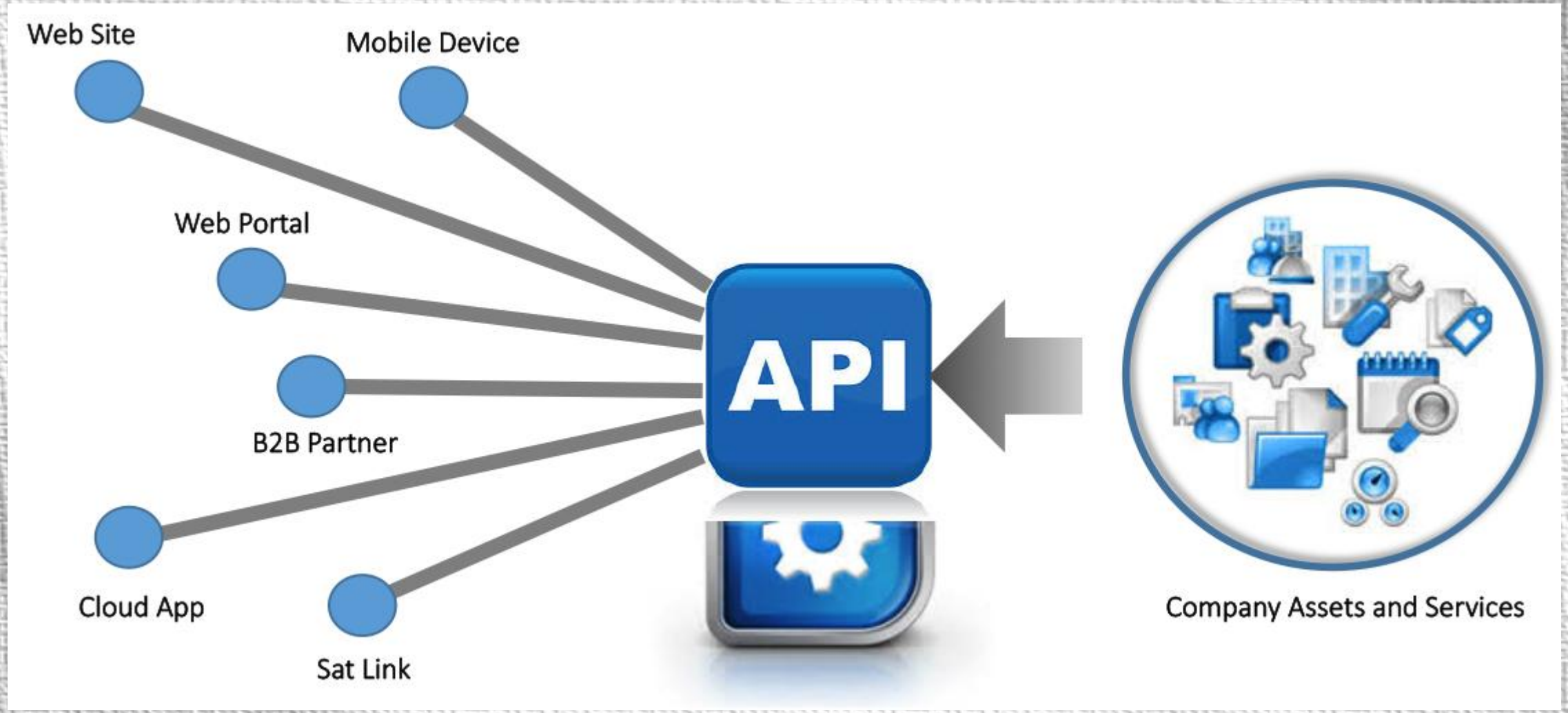
протокол HTTP



Запросы выполняются через HTTP, и ответы возвращаются через HTTP.

Веб-протокол является частью веб-сервиса: они независимы от языка и поэтому совместимы между различными платформами и системами. При документировании REST API не имеет значения, строят ли инженеры API с помощью Java, Ruby, Python или какого-либо другого языка. Запросы выполняются через HTTP, и ответы возвращаются через HTTP.

Рисунок ниже показывает общую модель REST API.


Web Site
Mobile Device
Web Portal
B2B Partner
Cloud App
Sat Link
API
Company Assets and Services

REST API model



REST API
(Java)

Request
URI: http://coolhomes.api.com/homes?limit=5
Method: GET

Request URI (HTTP)

Response (HTTP)

Response (JSON)

```
[
  {
    "home": "123"
    "location":"AZ"
    "price":"200k"
  },
  {
    "home":"456",
    "location": "CA"
    "price": "800K"
  }
]
```

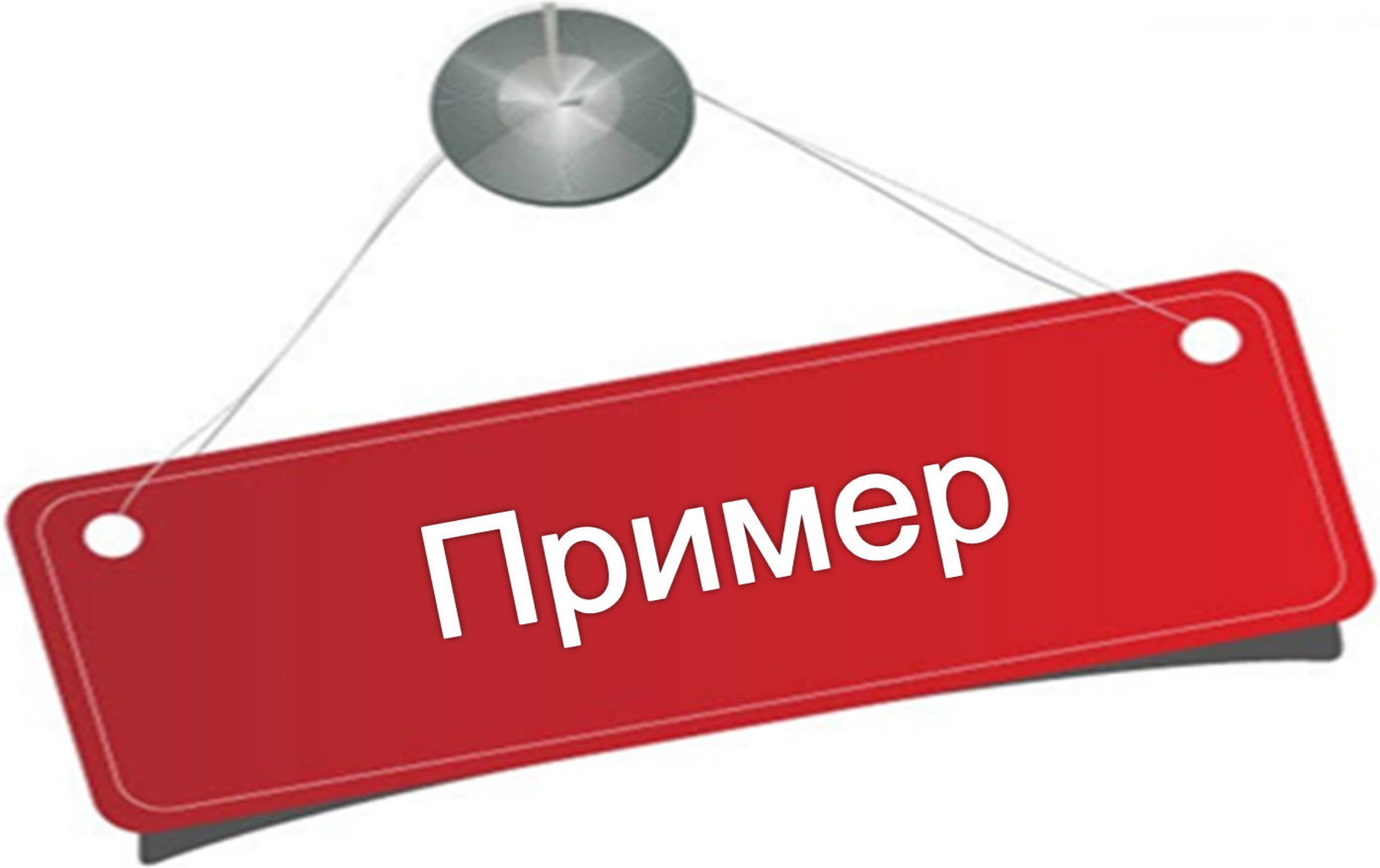


Application
(Ruby)

Как видно, между клиентом и сервером API существует запрос и ответ. Клиент и сервер могут быть основаны на любом языке, но HTTP - это протокол, используемый для передачи сообщения. **Этот шаблон запроса и ответа, по сути, является принципом работы API REST**.

Любой язык программирования, создающий запрос, будет по-своему отправлять веб-запрос и анализировать ответ на своем языке. Эти специфичные для языка функции отправки запросов и анализа ответов не являются частью REST API (хотя они могут быть предоставлены в прилагаемом SDK).

REST API не зависит от языка и обрабатывает входящую и исходящую информацию по HTTP.


Пример

Примеры использования веб-API

- **API в картографии** обычно используются для интеграции карты на веб-странице или в мобильном приложении.
- **API платежных сервисов** обычно используются компаниями, занимающимися e-commerce, для повышения гибкости процесса покупки, что приводит к расширению базы потенциальных клиентов.
- **API метеорологических служб** могут улучшить опыт пользователей спортивных приложений, поисковых систем и т. д. (`https://api.openweathermap.org/`)

Ниже приведено несколько примеров известных API, использующих различные протоколы и спецификации. Ознакомьтесь с их документацией, чтобы получить дополнительную информацию и ссылки.

- **Карты Google**. Ни для кого не секрет, что Google входит в число технологических гигантов, и они установили стандарты в том, как работают другие компании. Большинство веб-сайтов со встроенной картой используют API Google Карт.
- **Вулкан**. Vulkan — это кроссплатформенный API, который работает на уровне операционной системы. Он позволяет разработчикам создавать высококачественную графику в приложениях в режиме реального времени и обеспечивает связь между приложением и графическим процессором (GPU). Если вам интересно, ознакомьтесь с документацией Vulkan API.

- **Поиск рейсов Skyscanner**. Skyscanner — это метапоисковая платформа, которая позволяет путешественникам искать авиабилеты по лучшим ценам из своей базы данных. Кроме того, Skyscanner предоставляет своим аффилированным партнерам RESTful API, поддерживающий как XML, так и JSON в качестве форматов обмена данными. Для повышения безопасности они призывают партнеров использовать только протокол HTTPS для выполнения запросов.
- **Погодный API**. Это бесплатный поставщик информации о геолокации и погоде с множеством различных API, которые предоставляют доступ к прогнозу погоды, поиску IP-адресов, спорту, астрономии, геолокации и другим данным. Разработчики могут использовать HTTP или HTTPS для запроса JSON/XML RESTful API, используя при этом подробную документацию.

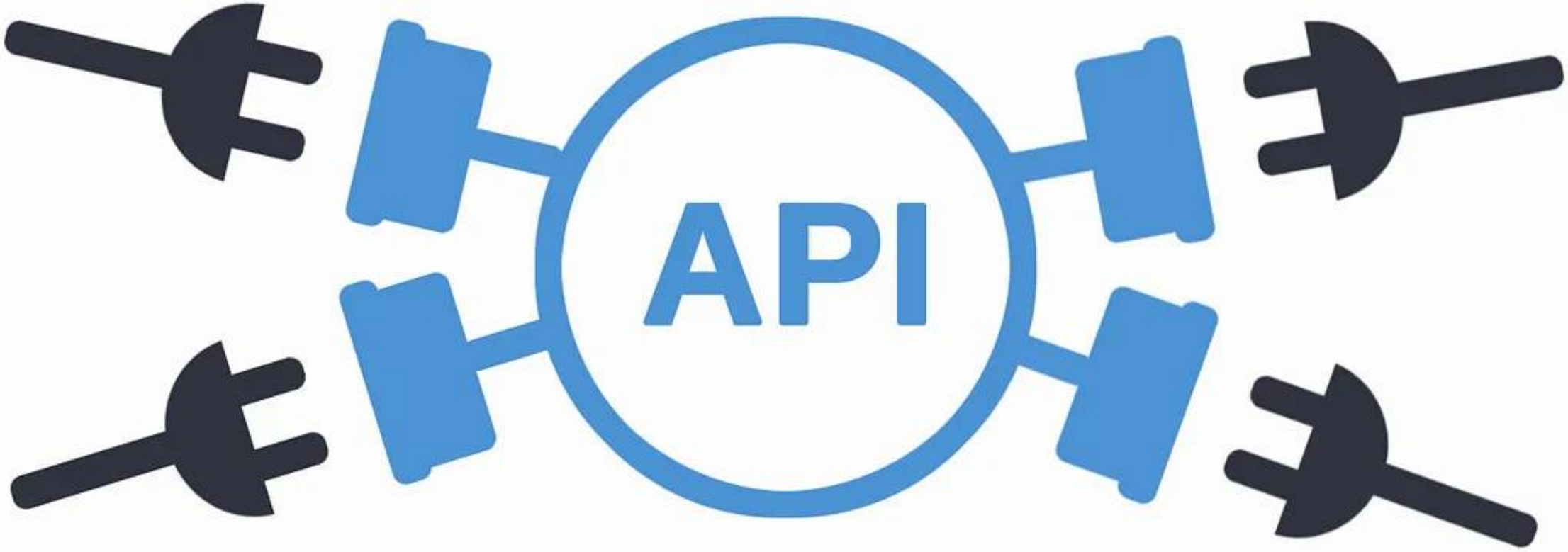
- **API Amadeus для путешествий**. Amadeus, еще одна крупная GDS, предлагает коллекцию REST и SOAP API, которая позволяет туристическим платформам искать и бронировать в более чем 400 авиакомпаниях, 150 000 отелях, 100 туроператорах, 90 железных дорогах, 75 компаниях по аренде автомобилей и т. д. API-интерфейсы из пакета самообслуживания выполняют несколько транзакций в ответ на один запрос, скрывая от разработчиков сложности процесса бронирования и ускоряя процесс интеграции.

Пример использования веб-API

https://api.openweathermap.org/

(22) API против SDK: в чем разница? - YouTube




API

API-шлюз

Представьте себя библиотекарем в огромной библиотеке с кучей книг и материалов. Ваша цель — облегчить посетителям доступ к информации и в то же время обеспечить ее сохранность. Вот тут-то и приходят на помощь шлюзы API.

Они подобны дружелюбным библиотекарям, которые помогают всем желающим найти нужные книги и ресурсы и для поддержания порядка просят каждого посетителя предъявить читательский билет. Эти библиотекари следят за тем, чтобы никто не брал слишком много книг одновременно и чтобы у каждого была возможность получить нужный документ. Когда кто-то запрашивает информацию, они сверяются со специальным справочником, чтобы убедиться, что дают правильные ответы. Кроме того, они помогают найти нужную информацию и следят за тем, чтобы она была корректной и понятной.

Стремительный рост количества нативных облачных приложений — более компактных, распределенных и предназначенных для работы в высокодинамичных средах, — сделал API-шлюзы незаменимыми посредниками при реализации цифровых инициатив.

Роль API-шлюзов

API-шлюз выступает в качестве центральной точки входа для внешних запросов и обеспечивает единый интерфейс для доступа к различным его компонентам. Он решает такие задачи, как маршрутизация запросов, аутентификация, авторизация, ограничение частоты запросов и сбор аналитических данных API. Снимая с индивидуальных компонентов эти заботы, API-шлюз упрощает разработку, повышает производительность и общую безопасность приложения.

Основные функции API-шлюза

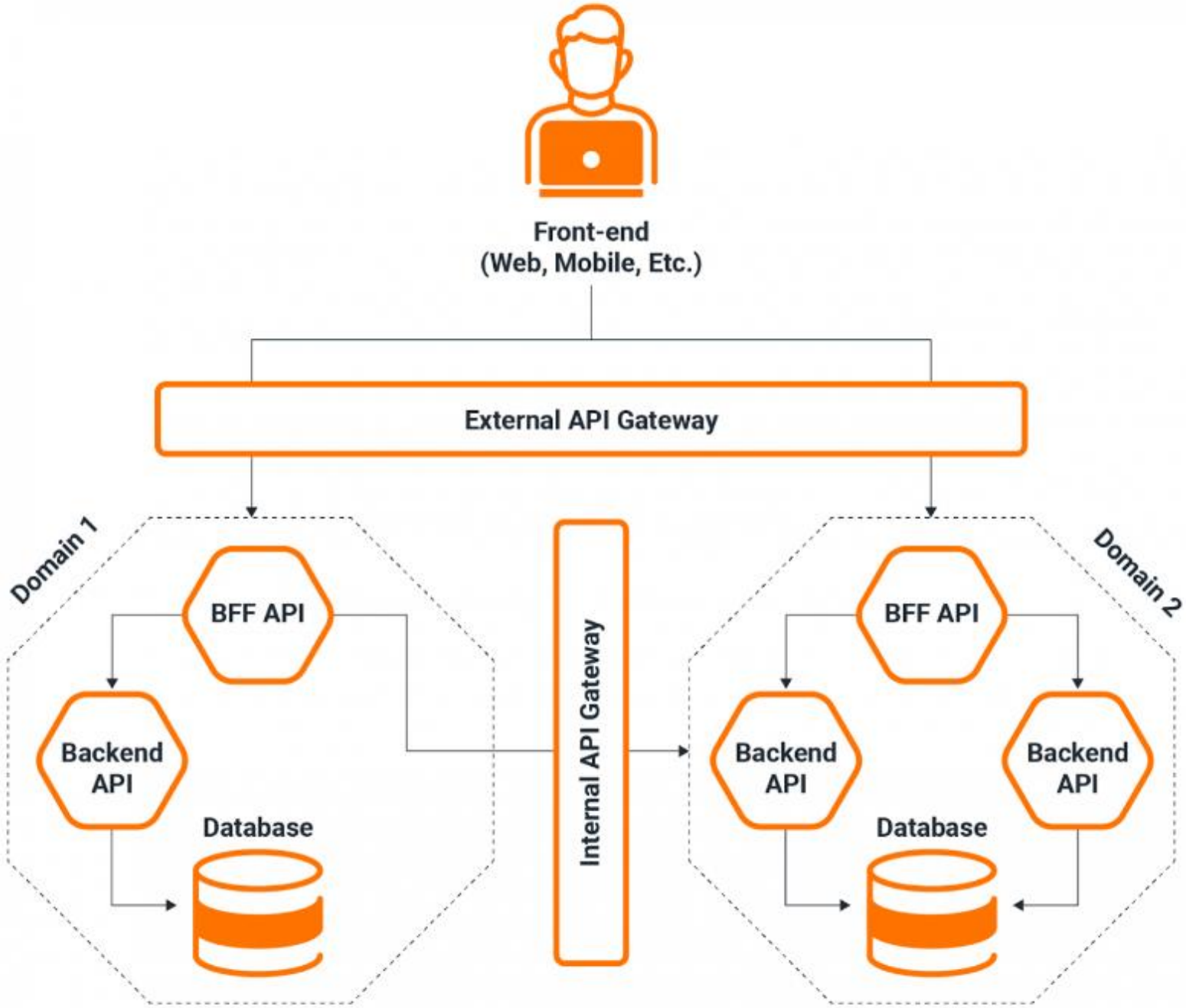
1) **Маршрутизация запросов**: Направляет запросы к нужным микросервисам.
2) **Аутентификация и авторизация**: Проверяет права доступа пользователей.
3) **Ограничение частоты запросов**: Предотвращает перегрузку системы.
4) **Сбор аналитики**: Отслеживает и анализирует использование API.
5) **Обеспечение безопасности**: Реализует политики защиты данных.

Роль в облачных приложениях

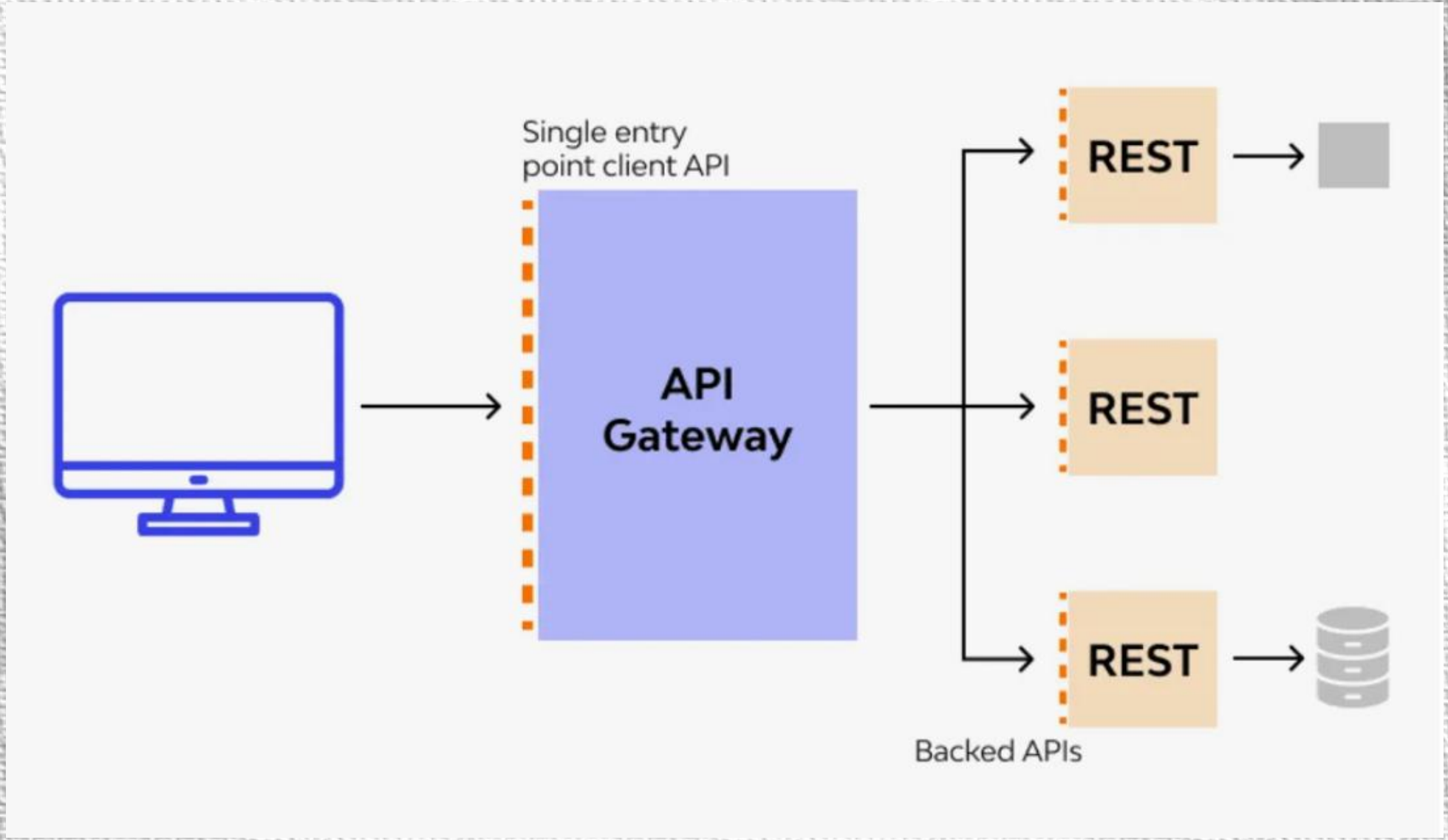
API-шлюзы особенно важны для нативных облачных приложений, которые состоят из множества микросервисов. Они обеспечивают взаимодействие между фронтендом, бэкендом и другими компонентами, такими как базы данных. Например, фронтенд отправляет запросы через API-шлюз, который перенаправляет их к соответствующим сервисам, таким как Backend For Frontend (BFF) или бэкенд-сервисы.

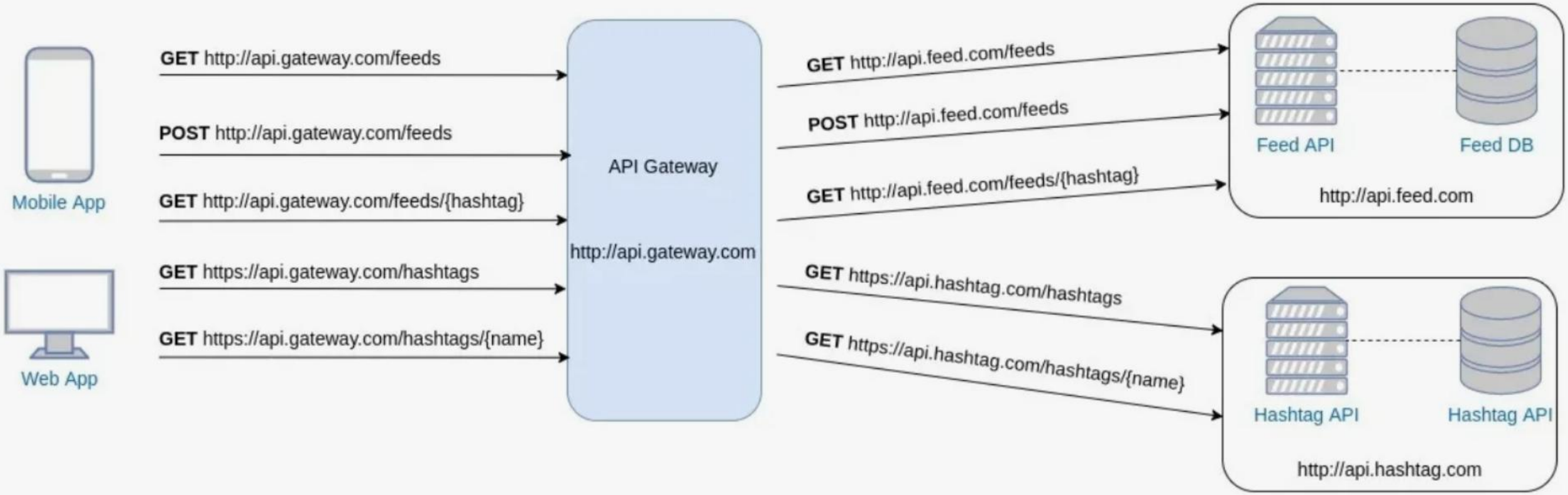
Организации с большим количеством сервисов обычно группируют их по бизнес-доменам. Для этого используются принципы доменно-ориентированного проектирования (domain-driven design, DDD). Когда сервисы в домене связываются с сервисами в других доменах, вызовы обычно проходят через API-шлюз. В таких случаях, как правило, применяются два API-шлюза: один для внутренних API, другой — для внешних.

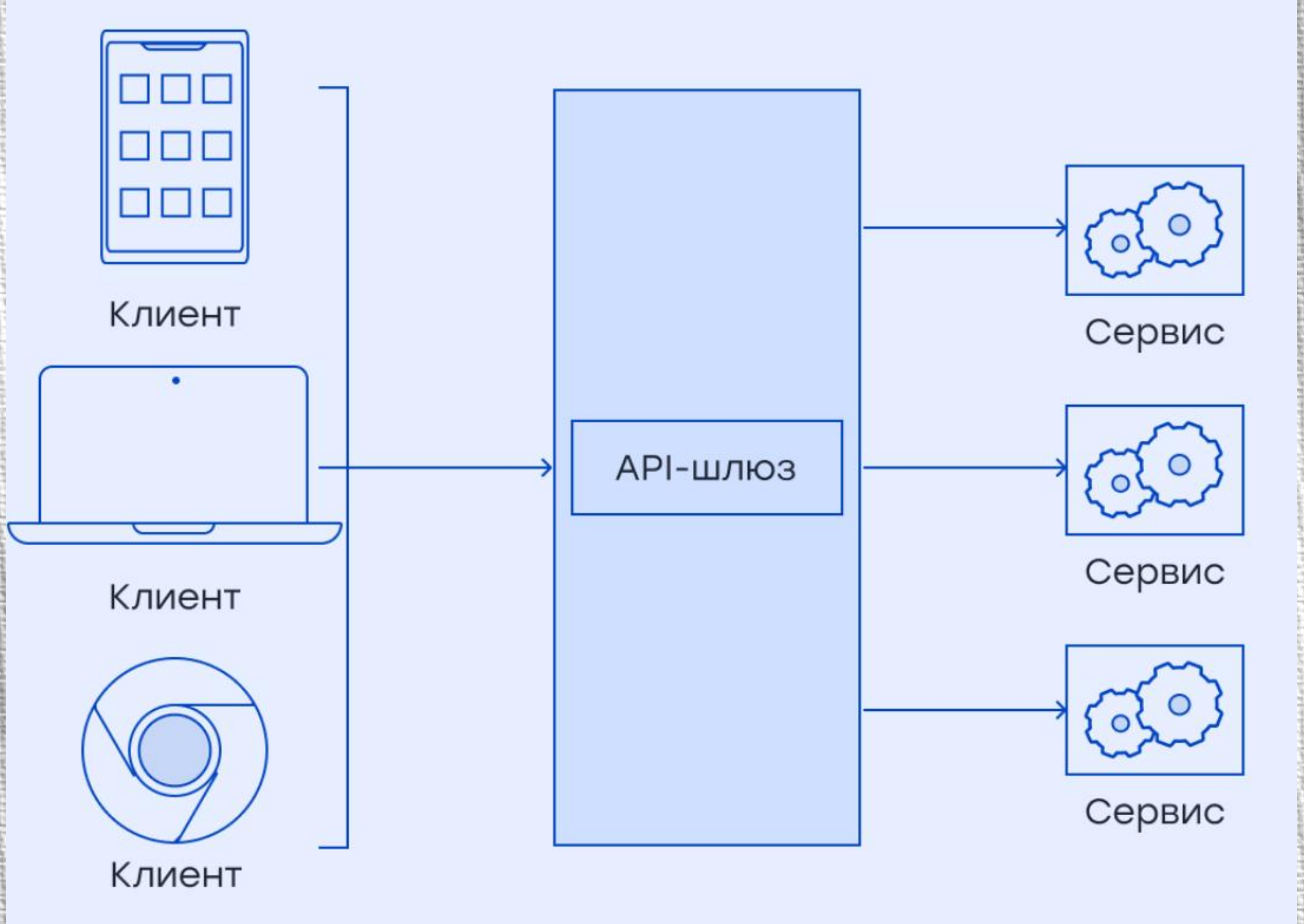
Шлюзы API в нативной облачной архитектуре



API-шлюз


Single entry
point client API
API
Gateway
REST
REST
REST
Backed APIs


Mobile App
Web App
GET http://api.gateway.com/feeds
POST http://api.gateway.com/feeds
GET http://api.gateway.com/feeds/{hashtag}
GET https://api.gateway.com/hashtags
GET https://api.gateway.com/hashtags/{name}
API Gateway
http://api.gateway.com
GET http://api.feed.com/feeds
POST http://api.feed.com/feeds
GET http://api.feed.com/feeds/{hashtag}
Feed API
Feed DB
http://api.feed.com
GET https://api.hashtag.com/hashtags
GET https://api.hashtag.com/hashtags/{name}
Hashtag API
Hashtag API
http://api.hashtag.com

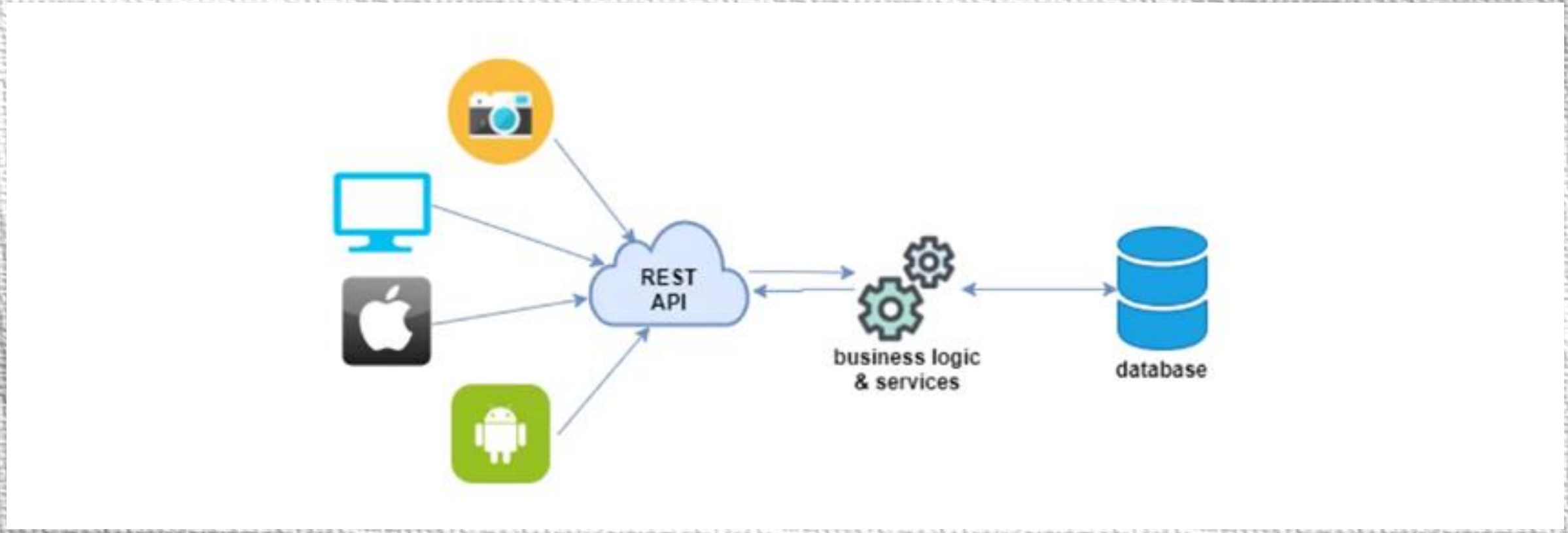

Клиент
Клиент
Клиент
API-шлюз
Сервис
Сервис
Сервис

API REST

- Что такое API
- Протокол
- Формат данных
- Аутентификация

Компьютеры общались и до REST, и все работало. Но сейчас в индустрии появился стандарт.

Способы взаимодействия компьютерных программ



Как сделать так, чтобы обращение к бизнес-логике было единообразным для всех клиентов?

Для решения данной задачи и существует программный интерфейс (на схеме это REST API).

Таким образом, API (Application programming interface) — это описание способов, с помощью которых одна компьютерная программа может взаимодействовать с другой.

API отвечает на вопрос «как можно обратиться к системе?» и включает в себя:

- Операцию, которую мы можем выполнить;
- Данные, которые поступают на вход;
- Данные на выходе (данные или сообщение об ошибке).

REST API — это программный интерфейс, построенный по принципам архитектуры REST.

Вызов API — это запрос, который клиентское приложение пересылает на сервер. Он содержит

- выполняемые операции (например, GET для получения ресурса или POST для отправки данных),
- данные аутентификации — например, ключ API, который идентифицирует клиента,
- дополнительные параметры, а также
- адрес назначения — URL-адрес конечной точки API.

Конечная точка API похожа на вход в место, где находится искомый ресурс (данные или функция). Эндпоинт проверяет API-ключ и, в случае одобрения, отправляет ответ с информацией о статусе операции (ошибка или успех) и запрошенных ресурсах.

REST – это стиль архитектуры программного обеспечения для распределенных систем, таких как World Wide Web, который, как правило, используется для построения веб-служб.

Термин REST был введен в 2000 году Роем Филдингом, одним из авторов HTTP-протокола. Системы, поддерживающие REST, называются RESTful-системами.

Рой Филдинг, автор REST

На фото Рой Филдинг, который в 2000 году защитил диссертацию «Архитектурные стили и дизайн сетевых программных архитектур» и тем самым ввел термин REST. Более того, он придумал HTTP и, по сути, является одним из основателей Интернета.



REST - стиль, а не стандарт

REST — это общий набор принципов, характерных не только для Web. Он может быть применен к другим видам сетей, таким как встроенные системы. REST — не протокол, так как он не задаёт деталей реализации.



Технология REST получила широкое распространение в Web как более простая альтернатива Web-сервисам, основанным на SOAP и языке Web Services Description Language (WSDL). Фактически REST оказала настолько большое влияние на Web, что практически вытеснила дизайн интерфейса, основанный на SOAP и WSDL, из-за значительного более простого стиля проектирования.

В общем случае REST является очень простым интерфейсом управления информацией без использования каких-то дополнительных внутренних прослоек.

RESTful сервер должен прятать от клиента как можно больше деталей своей реализации. Клиенту не следует знать о том, какая СУБД используется на сервере или сколько серверов в данный момент обрабатывают запросы и прочие подобные вещи. Организация правильного разделения функций важна для масштабирования если ваш проект начнет быстро набирать популярность.

В архитектурном стиле нет ограничений на формат сообщения

API REST могут использовать любой формат сообщений, который хотят использовать разработчики API, включая XML, JSON, Atom, RSS, CSV, HTML и другие.

Несмотря на разнообразие параметров формата сообщений, большинство API REST используют JSON (нотацию объектов JavaScript) в качестве формата сообщений по умолчанию. Они используют JSON, потому что он обеспечивает легкий, простой и более гибкий формат сообщений, который увеличивает скорость связи. Облегченная природа JSON также позволяет выполнять сценарии мобильной разработки и легок для парсинга в Интернете с помощью JavaScript.

REST фокусируется на ресурсах, доступ к которым осуществляется через URL

Другой отличительной чертой REST является то, что API-интерфейсы REST фокусируются на ресурсах (то есть вещах, а не действиях) и способах доступа к ресурсам. Ресурсы, как правило, являются разными типами информации. Вы получаете доступ к ресурсам через URL (Uniform Resource Locators), так же как переход к URL-адресу в вашем браузере позволяет подключиться к информационному ресурсу. URL-адреса сопровождаются методом, который указывает, как вы хотите взаимодействовать с ресурсом.

Общие методы включают GET (чтение), POST (создание), PUT (обновление) и DELETE (удаление). Конечная точка обычно включает параметры запроса, которые определяют более подробную информацию о представлении ресурса, который нужно увидеть. Например, можно указать (в параметре запроса) ограничение на отображение 5 экземпляров ресурса.

Вот пример, как выглядит конечная точка:

```
http://apiserver.com/homes?limit=5&format=json
```

Конечная точка показывает весь путь к ресурсу. Однако в документации вы обычно разделяете этот URL на более конкретные части:

- Базовый путь (базовый URL или хост) относится к общему пути для API. В приведенном выше примере базовый путь - http://apiserver.com;
- Отношение конечной точки к конечному пути этой точки. В приведенном примере это /homes;
- ?limit=5&format=json часть конечной точки содержит параметры строки запроса для этой точки.

В приведенном выше примере конечная точка получит ресурс «homes» и ограничит результат до 5 домов. Будет возвращен ответ в формате JSON.

Можно иметь несколько конечных точек, которые ссылаются на один и тот же ресурс. Вот один из вариантов:

http://apiserver.com/homes/{home id}

Приведенный выше URL-адрес может быть конечной точкой, которая извлекает домашний ресурс, содержащий определенный идентификатор. То, что передается обратно с сервера на клиент, называется «представлением» ресурса. Ресурс может иметь много разных представлений (показывая все дома или дома, которые соответствуют определенным критериям, или дома в определенном формате и т.д.), Но здесь мы хотим видеть дом с определенным идентификатором.

JSON является одним из многих возможных представлений ресурсов. Таким образом, XML, двоичные форматы или любой другой формат сериализации, который мы можем придумать, в принципе, также могут быть использованы как представление в RESTful-сервисе.

Требования к архитектуре REST

REST – это гибрид, основанный на трех архитектурных стилях, которые добавляют друг друга, но все **базируются на отделении сервера от клиента.**



❶ Модель клиент-сервер

RESTful-система должна производить операции в клиент-серверной модели, даже если компонент периодически ведёт себя то, как клиент, то, как сервер.

Альтернатива клиент-серверной архитектуре, построенная без REST — это интеграция, основанная на событиях. В этой модели каждый компонент непрерывно передает события, перехватывая соответствующие события из других компонентов. **В ней нет взаимодействия один-к-одному, только передача и перехват**. REST требует взаимодействия один-к-одному, поэтому архитектура, основанная на событиях, не будет удовлетворять требованиям RESTful.

В одной из глав Филдинг обозначил архитектурные принципы, которые преследуют единственную цель - **унификация и стандартизация общения между компьютерами внутри сети**.

Это важно, если мы хотим построить систему, которую будет легко масштабировать и поддерживать.

Согласитесь, удобно, когда мы можем отсоединить один элемент, как кубик лего, и поставить на его место совершенно другой, а вся эта конструкция продолжит работать.

Фундаментальный принцип REST

«Разделение клиента и сервера позволяет каждой части эволюционировать независимо, обеспечивая масштабируемость, переносимость и возможность независимой разработки компонентов системы.»

Это разделение создает основу для современной веб-экосистемы, где множество разнообразных клиентов могут взаимодействовать с единым, централизованным источником данных и бизнес-логики.

❷ Отсутствие состояния

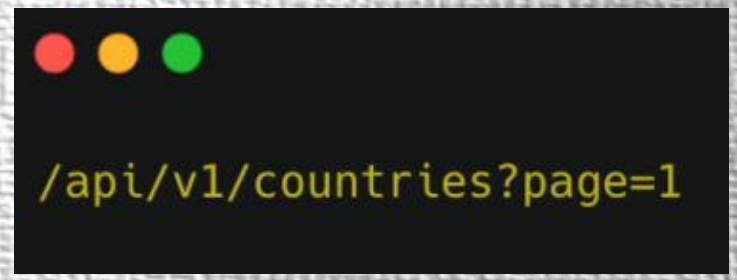
От второго архитектурного стиля мы позаимствуем отсутствие записи состояния клиента.

Другими словами, сервер не должен запоминать, какие запросы клиент уже сделал и в каком состоянии он находится сейчас.

Каждый новый запрос должен быть сформирован так, будто до этого клиент с сервером не общались и это их первый контакт.

Проще говоря, RESTful сервер не должен отслеживать, хранить и тем более использовать в работе текущую контекстную информацию о клиенте. С другой стороны клиент должен взять эту задачу на себя. Другими словами не заставляйте сервер помнить состояние мобильного устройства, использующего API.

Представим, что у нас есть список всех стран, разделенный на странице. На клиенте мы запросили первую страницу и отрисовали весь необходимый интерфейс.



Чтобы загрузить следующую страницу, мы должны четко попросить страницу номер два.



Мы не можем запросить просто следующую страницу, потому что сервер не запоминает, какая страница была предыдущей.



Независимость от состояния означает, что данные, возвращаемые определенным вызовом API, не должны зависеть от вызовов, сделанных ранее.

Клиент же со своей стороны, может (должен) помнить параметры, сгенерированные на сервере при обращении к нему и использовать их для последующих вызовов API, если потребуется.

Однако, важно отметить, что REST не запрещает серверу хранить состояние вообще. Речь идет о том, что состояние сессии (информация о клиенте, которая должна сохраняться между запросами) не хранится на сервере.

Вместо этого, клиент должен включать в каждый запрос всю необходимую информацию (например, токен аутентификации, параметры, которые обычно хранились бы в сессии).

Пример: если пользователь аутентифицирован, то каждый запрос должен содержать токен (например, в заголовке Authorization), а сервер проверяет этот токен при каждом запросе, не храня информацию о том, что пользователь вошел в систему.

Противоположность — stateful (с сохранением состояния), как, например, в сессиях, где сервер хранит информацию о клиенте и требует, чтобы последующие запросы приходили на тот же сервер, который обрабатывал предыдущие (если используется несколько серверов, то состояние сессии должно быть общим для всех серверов, что усложняет архитектуру).

Итак, требование «отсутствия состояния» в RESTful-системе означает, что сервер не хранит состояние сессии клиента, и каждый запрос должен быть самодостаточным для его обработки.

Преимущества stateless-подхода

Масштабируемость

- Любой сервер может обработать любой запрос

  Client → Load Balancer → [Server A, Server B, Server C]

- Неважно, на какой сервер попадет запрос

Надежность

- При падении сервера не теряются данные сессий
- Клиенты могут повторно отправлять запросы на другие серверы

Простота кэширования

- Кэш может хранить ответ, так как он зависит только от данных в запросе.

Что НЕ означает "отсутствие состояния"

Сервер МОЖЕТ хранить:

- Данные приложения (базы данных, файлы)
- Кэши (для повышения производительности)
- Конфигурацию

Сервер НЕ ДОЛЖЕН хранить:

- Состояние клиентской сессии
- Информацию о предыдущих запросах клиента
- Временные данные, необходимые для обработки следующих запросов

Золотое правило REST

"Если серверу нужна какая-либо информация о клиенте для обработки запроса, эта информация должна быть передана в самом запросе, а не подразумеваться из предыдущих взаимодействий."

Этот принцип делает REST-сервисы предсказуемыми, надежными и легко масштабируемыми, что особенно важно в современных распределенных системах и микросервисных архитектурах.

❸ Кэшируемость

Возвращаясь к примеру выше, отметим, что список стран не так часто меняется, а значит запросы будут повторяться.
Поэтому REST позаимствовал у третьего стиля - кэшируемость. Сервер помечает свой ответ как кэшируемый, и клиент сохраняет его у себя в памяти.
В следующий раз, когда клиенту нужны будут эти данные, то вместо запроса на сервер он возьмет их из хранилища.
Это позволяет уменьшить общее число запросов, снизить нагрузку на сервер и, соответственно, повысить быстродействие программы.

Зачем нам это нужно? Это нужно и полезно, если у сервера часто запрашивают одинаковую информацию. Например, кэширование активно используется на новостных сайтах или в соцсетях (на веб-ресурсах, к которым происходит много обращений).

Все ответы сервера должны помечаться как кэшируемые или некэшируемые. Кэширование происходит, когда клиент сохраняет ответы, полученные ранее от сервера. Когда эти данные нужны снова, кэширование может избавить от полного прохода данных по сети.

Возможность кэшировать существует благодаря самодостаточным сообщениям. Клиенту не нужно беспокоиться о том, что случайно закэшируется только часть необходимой информации, а другие части потеряются.

❹ Единообразие интерфейса

И вот мы подошли к самому ключевому принципу REST, который и отличает этот архитектурный стиль от всех других. Этим принципом мы вводим ограничения, благодаря которым интерфейс общения между клиентом и сервером становится единообразным.

Этот принцип является одним из ключевых ограничений архитектурного стиля REST.

А это важно, если мы все еще хотим иметь возможность поддерживать и расширять систему без головной боли. Этот принцип называется **унификация или единообразие интерфейса**. Первое ограничение связано с идентификацией ресурса.

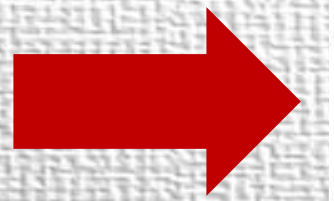
1-е ограничение интерфейса: определение ресурсов

В терминологии REST ресурсом может быть что угодно: почта, документ, картинка или, например, информация о юзере.

Самое главное то, что каждый ресурс должен иметь стабильный уникальный идентификатор или по- другому URI (ю эр ай).

Допустим, у нас есть статьи в базе данных. Для REST эта сущность будет являться ресурсом, а значит нам нужен ее идентификатор.

```
/getArticles
/editArticles
/updateArticles
/deleteArticles
```



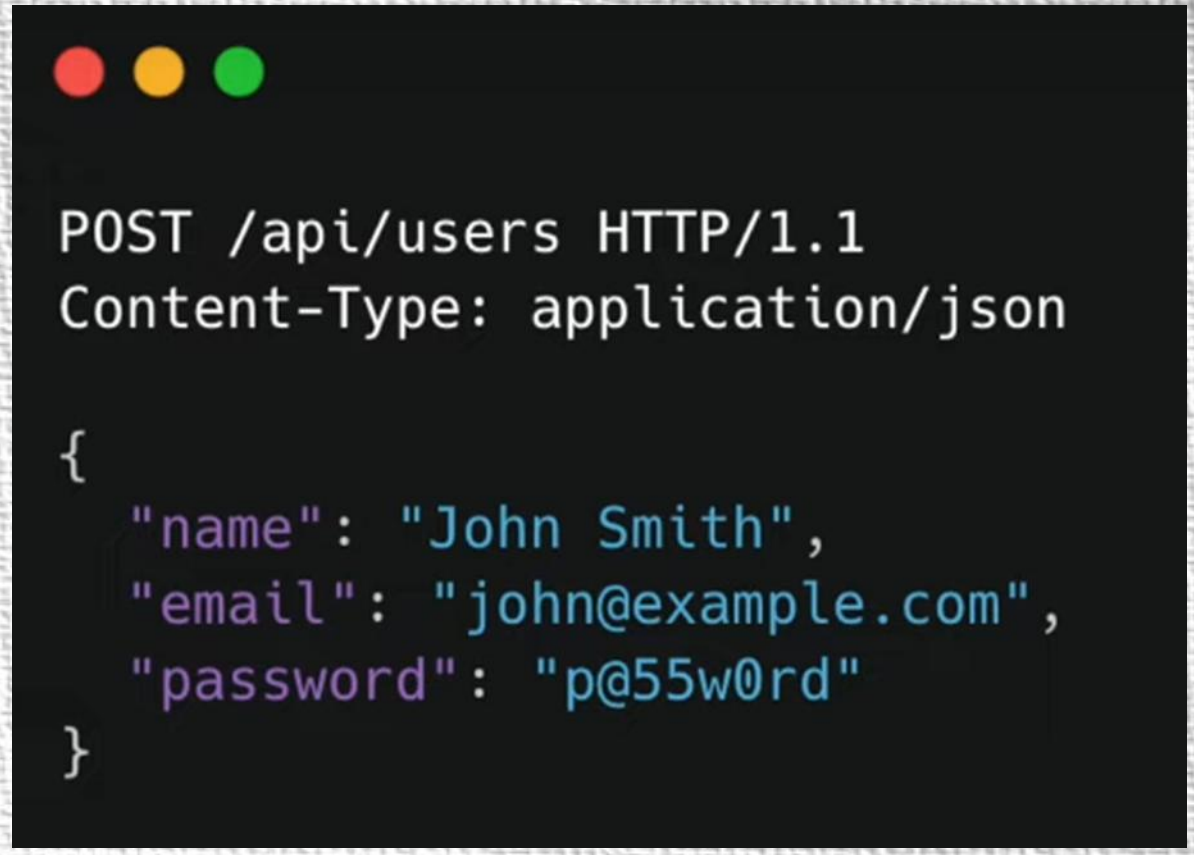
```
/articles GET
/articles POST
/articles PUT
/articles DELETE
```

Вместо четырех разных URI мы должны использовать один единый. А уже за манипуляцию данными будет отвечать метод запроса!

2-е ограничение интерфейса: управление ресурсами через представления

Так как клиент не может напрямую менять данные, он должен предоставить серверу свое видение данных, то, как он их представляет.

То есть вместо того, чтобы говорить «*поменять такое-то поле у такого-то ресурса*», мы отправляем измененный ресурс целиком, как бы говоря: вот таким я бы хотел его видеть.



Это ограничение называется манипуляцией ресурсом через представления.

Клиенты взаимодействуют с ресурсами через представления (например, JSON, XML), которые передаются в HTTP-запросах и ответах.

Представление — это состояние ресурса в определенный момент времени, переданное в определенном формате (JSON, XML и т.д.).

Суть принципа

Клиент никогда не работает с самим ресурсом напрямую, а только с его представлениями — сериализованными версиями, которые передаются по сети.

Зачем это нужно?

✓ **Разделение ответственности**: сервер управляет ресурсами, клиент — представлениями.

✓ **Гибкость:** один и тот же ресурс может быть представлен в разных форматах (JSON, XML, HTML) для разных клиентов.

✓ **Сокрытие внутренней реализации:** клиент не знает, как ресурс хранится или обрабатывается на сервере, он работает только с представлением.

Пример:

Ресурс "пользователь" может быть представлен в виде JSON для мобильного приложения и в виде HTML для браузера.

Как это работает?

- Клиент запрашивает ресурс, указывая желаемое представление (через заголовки Accept, Content-Type).
- Сервер возвращает представление ресурса, которое может быть не полным (например, только некоторые поля) или преобразованным.
- Клиент может модифицировать представление и отправить его обратно для обновления ресурса (например, через PUT или PATCH).

Важно: представление — это не сам ресурс, а его проекция. **Ресурс может иметь несколько представлений.**

Пример в коде

Запрос:

```
GET /users/123 HTTP/1.1
Accept: application/json
```

Ответ:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
"id": 123,
"name": "John Doe",
"email": "john@example.com"
}
```

Теперь клиент может изменить имя и отправить обратно:

text

```
PUT /users/123 HTTP/1.1
Content-Type: application/json
{
  "id": 123,
  "name": "John Smith",
  "email": "john@example.com"
}
```

Сервер принимает это представление и обновляет ресурс.

Также представление может включать гипермедийные ссылки (HATEOAS), что позволяет клиенту discover действия, которые можно выполнить с ресурсом.

Пример с гиперссылками:

```
{
"id": 123,
"name": "John Doe",
"_links": {
"self": { "href": "/users/123" },
"orders": { "href": "/users/123/orders" }
}
}
```

Таким образом, управление ресурсами через представления позволяет строить гибкие, слабосвязанные системы, которые могут эволюционировать независимо.

Преимущества подхода

Сокрытие внутренней реализации

Клиенту неважно, как хранится ресурс:

- Реляционная БД (MySQL)
- Документная БД (MongoDB)
- Файловая система
- Внешний микросервис

Эволюция API без ломающих изменений

Версия 1.0

```
{
  "id": 123,
  "name": "John"
}
```

Версия 1.1 - добавляем поле

```
{
  "id": 123,
  "name": "John",
  "email": "john@example.com"  // новое поле
}
```

Старые клиенты игнорируют новое поле
Обратная совместимость сохранена

Гибкая трансформация данных

javascript

Сервер может адаптировать представление под контекст

```
function getUserRepresentation(user, context) {
  const base = {
    id: user.id,
    name: user.name
  };

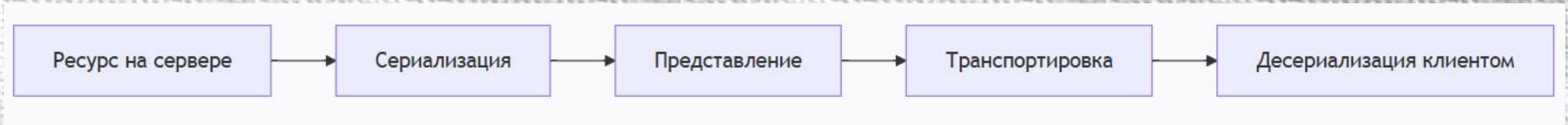
Добавляем поля в зависимости от прав доступа
  if (context.isAdmin) {
    base.email = user.email;
    base.lastLogin = user.lastLogin;
  }
```

Добавляем гипермедийные ссылки

```
    base._links = {
      self: `/users/${user.id}`,
      orders: `/users/${user.id}/orders`
    };

    return base;
  }
```

Жизненный цикл представления
От ресурса к представлению



Фундаментальный смысл

Управление через представления превращает REST из простого транспортного протокола в мощную абстракцию, где:

✅ **Сервер полностью контролирует свои данные и решает, что показывать**

✅ **Клиенты становятся независимыми от внутренней реализации**

✅ **Система получает возможность эволюционировать без ломающих изменений**

✅ **Появляется гибкость в форматах данных и уровнях детализации**

Этот принцип создает прочную основу для построения масштабируемых, поддерживаемых и интероперабельных веб-систем, которые могут развиваться десятилетиями.

3-е ограничение интерфейса: самодостаточные сообщения

Еще одно немаловажное ограничение звучит как самоописываемые сообщения (Self-Descriptive Messages).

В целом из названия понятно, что сообщение должно содержать в себе всю необходимую информацию, чтобы получатель понял и интерпретировал его.

Не должно быть ситуации, когда в первом сообщении содержатся данные, а во втором инструкции, как с этими данными взаимодействовать. То есть в отдельной документации или другом сообщении не должно быть дополнительной информации.

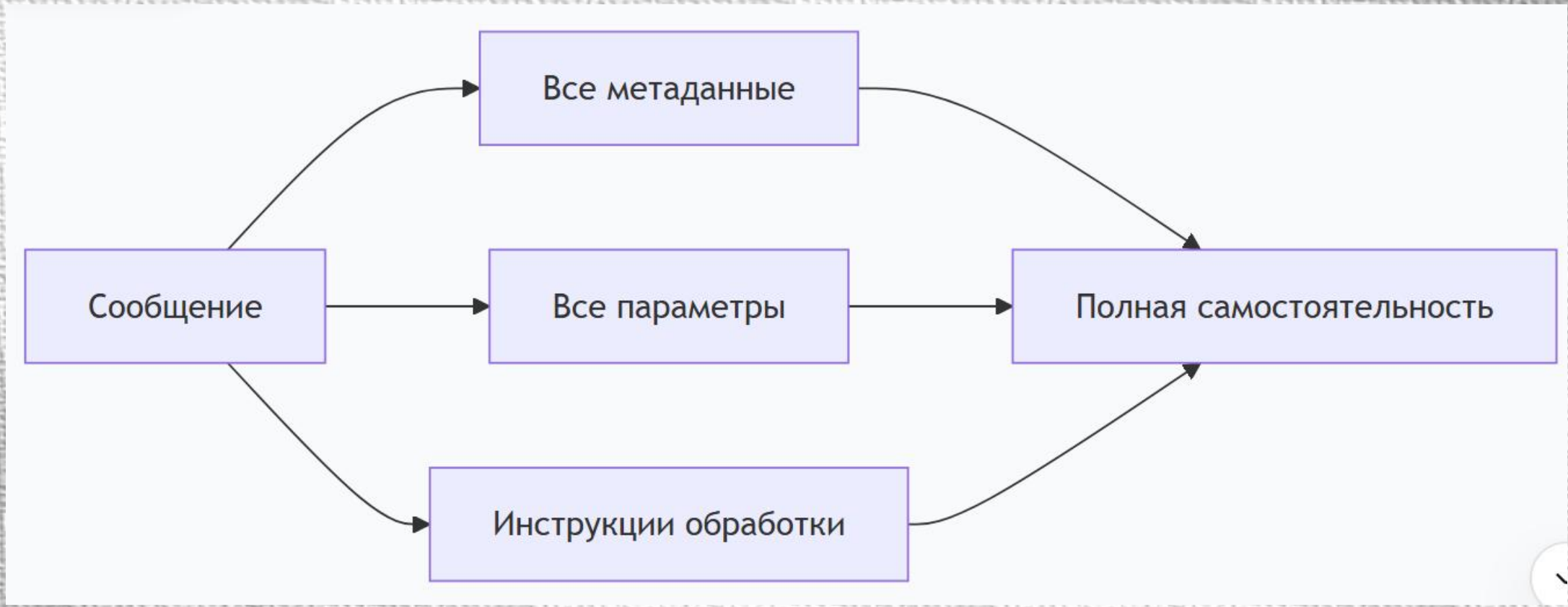
Например, когда мы вводим в строку браузера адрес **google.com**, браузер, то есть клиент посылает сообщение на сервер с вот таким содержанием:

```
GET / HTTP/2
Host: www. google.com
```

Так мы говорим серверу, что хотим получить корневой ресурс сайта. Этой информации достаточно, чтобы сервер вернул нам в ответ вот такое сообщение:

```
HTTP/2 200 OK
Content-Type: text/html

<!DOCTYPE html>
<html>
  <head>
    <title>Google</title>
  </head>
  </body>
    <img src="/images/logo.png" />
    <form action="/search" method="GET">
      <input type="text" name="q"/>
      <button type="submit">Search</button>
    </form>
    <a href="/help">Need help?</a>
  </body>
</html>
```


Все метаданные
Сообщение
Все параметры
Полная самостоятельность
Инструкции обработки

Что делает сообщение самодостаточным?

Обязательные компоненты запроса

http

```
# Метод + URI - ЧТО делать
GET /api/orders/456
# Заголовки - КАК обрабатывать
Authorization: Bearer token123     # Кто
Accept: application/json           # В каком формате ответ
Accept-Language: en-US             # Язык
Content-Type: application/json     # Формат тела
Cache-Control: no-cache            # Правила кэширования
If-None-Match: "abc123"            # Условия выполнения
```

Обязательные компоненты ответа

```
http
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Language: en-US
Cache-Control: max-age=3600
ETag: "xyz789"
Last-Modified: Wed, 21 Oct 2024 07:28:00 GMT

{
  "id": 456,
  "status": "processed",
  "_links": {
    "self": "/api/orders/456",
    "cancel": "/api/orders/456/cancel"
  }
}
```

4-е ограничение интерфейса: гипермедиа

Гипермедиа можно представить как набор гиперссылок, причем это может быть как html- документ со ссылками, так и JSON-объект со списком доступных ресурсов для запроса.

```
<div class="pagination_section">
    <a href="/home"><< Previous</a>
    <a href="/page/1" title="Algorithm">1</a>
    <a href="/page/2" title="DataStructure">2</a>
    <a href="/page/3" title="Languages">3</a>
    <a href="/page/4" title="Interview" class="active">4</a>
    <a href="/page/5" title="practice">5</a>
    <a href="/page/1">Next >></a>
</div>
```

```
{
    "first": "https://example.com/api/items?page=1",
    "last": "https://example.com/api/items?page=9",
    "next": "https://example.com/api/items?page=4",
    "prev": "https://example.com/api/items?page=2"
}
```

Основная идея этого ограничения в том, что сервер должен подсказывать клиенту, какие следующие шаги он может предпринять дальше.

На примере предыдущего сообщения сервер нам ясно указывает, что следующий наш запрос может быть `google.com/search` с параметром **q**, если пользователь отправит форму или `google.com/help`, если пользователь нажмет на ссылку **Need help.**

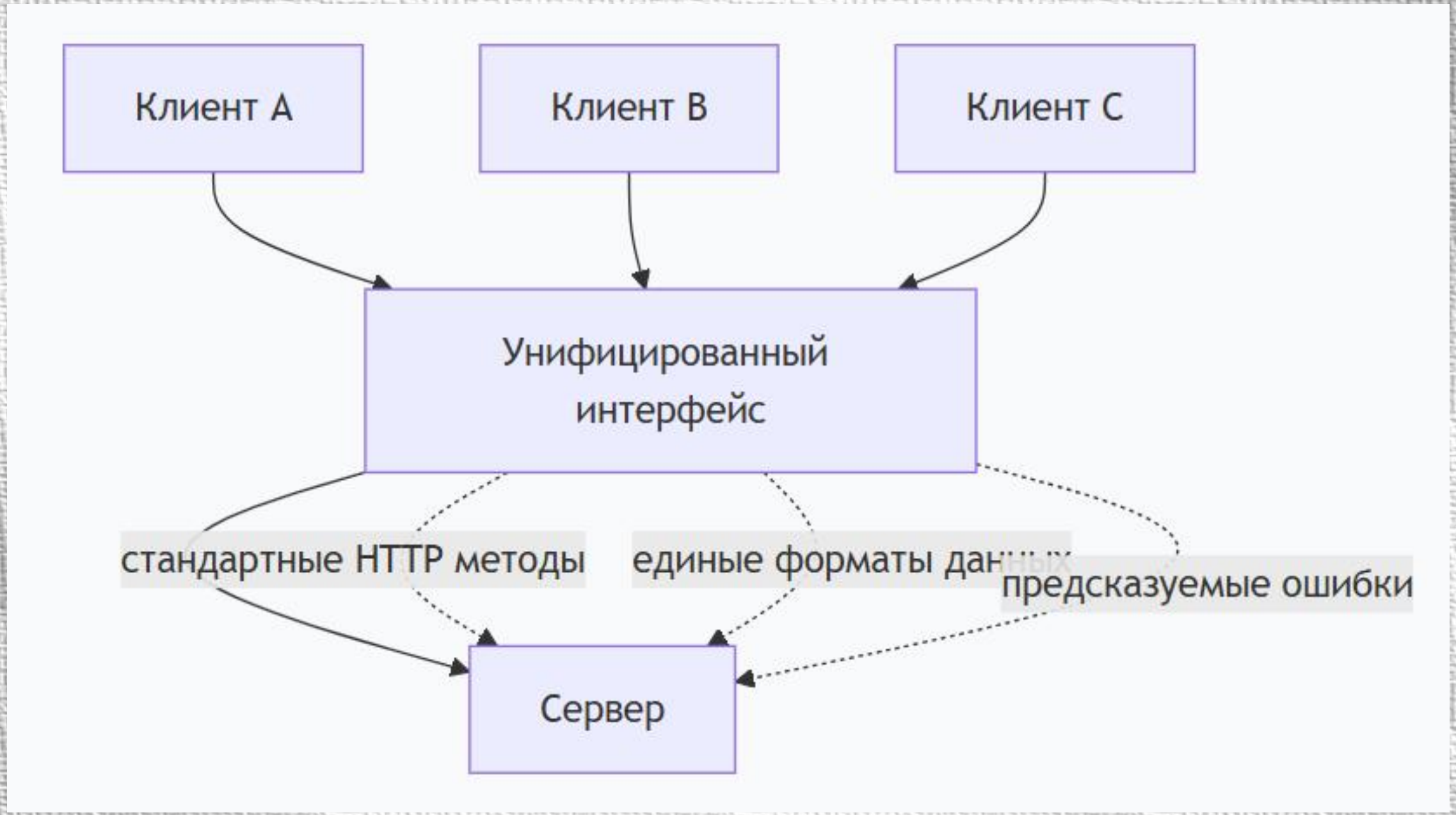
```
HTTP/2 200 OK
Content-Type: text/html

<!DOCTYPE html>
<html>
  <head>
    <title>Google</title>
  </head>
  </body>
    <img src="/images/logo.png" />
    <form action="/search" method="GET">
      <input type="text" name="q"/>
      <button type="submit">Search</button>
    </form>
    <a href="/help">Need help?</a>
  </body>
</html>
```

Идентификатор ресурса, манипуляция через представления, самоописываемые сообщения и гипермедиа как двигатель состояния вместе образуют принцип унификации интерфейса.

Суть принципа унификации интерфейса

Единый, стандартизированный способ взаимодействия со всеми ресурсами системы, **вместо специализированных API для каждого типа данных или операции**.


Клиент A
Клиент B
Клиент C
Унифицированный интерфейс
стандартные HTTP методы
единые форматы данных
предсказуемые ошибки
Сервер

Зачем вводится унификация интерфейса?

1. Упрощение системы и разделение ответственности

- Клиенты и серверы могут развиваться независимо, при условии, что интерфейс остается неизменным.
- Сервер может менять внутреннюю реализацию, не затрагивая клиентов, и наоборот.

2. Повышение переносимости и совместимости

- Стандартизированный интерфейс позволяет легко интегрировать разнородные системы.
- Разные клиенты (веб, мобильные, сторонние приложения) могут использовать один и тот же API.

3. Улучшение масштабируемости

- Единообразие позволяет прокси-серверам и кэшам эффективно работать, так как они понимают семантику запросов.

4. Упрощение клиентских приложений

- Клиенты могут быть более простыми, так как они используют единый подход к взаимодействию с сервером

Как реализуется унификация интерфейса в REST?

Использование HTTP методов единообразно

- GET /users/123 # Получить пользователя с id=123
- POST /users # Создать нового пользователя
- PUT /users/123 # Обновить пользователя с id=123
- DELETE /users/123 # Удалить пользователя с id=123

Использование кодов состояния HTTP

- 200 OK # Успешный запрос
- 201 Created # Ресурс создан
- 400 Bad Request # Неверный запрос
- 404 Not Found # Ресурс не найден
- 500 Internal Server Error # Ошибка сервера

Гипермедиа (HATEOAS)

json

```
{
  "user": {
    "id": 123,
    "name": "John Doe",
    "links": [
      { "rel": "self", "href": "/users/123" },
      { "rel": "orders", "href": "/users/123/orders" }
    ]
  }
}
```

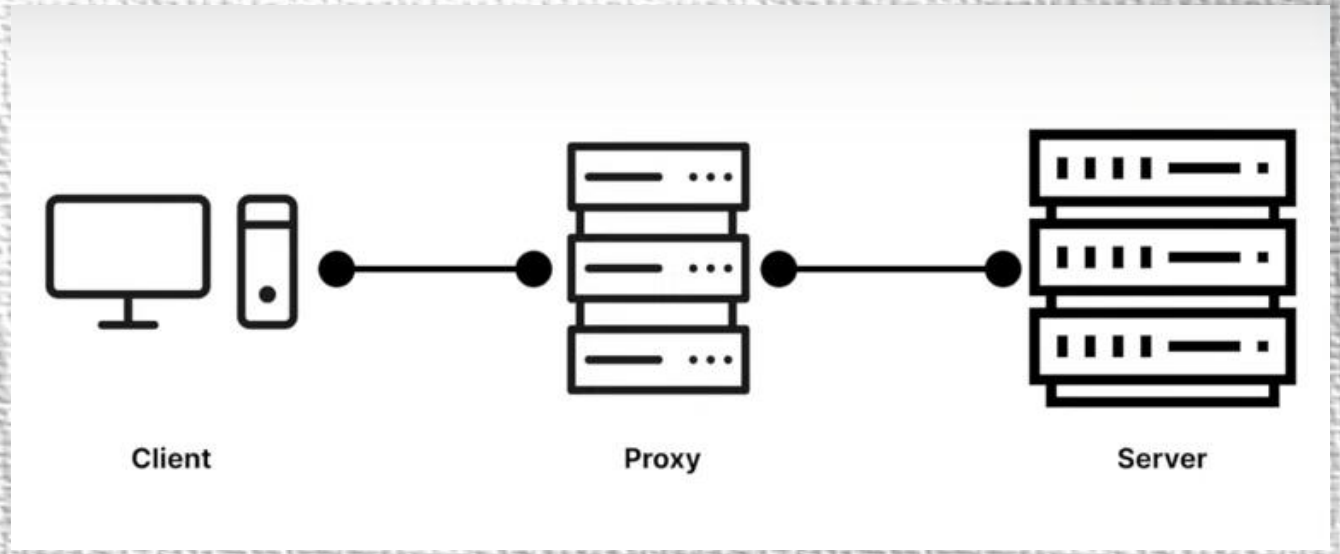
5 Система слоёв

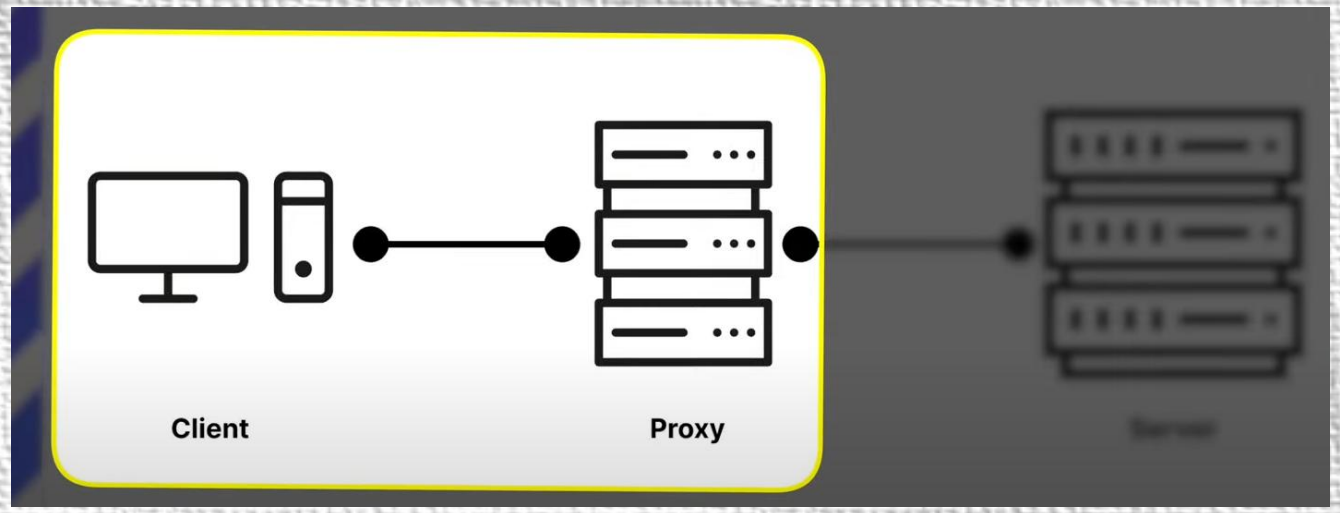
Принцип многоуровневости системы говорит нам, что в системе могут существовать не только уровни клиента и сервера. Если мы рассмотрим систему целиком, то она может иметь любое количество слоев.

Скажем, между клиентом и сервером может находиться дополнительный прокси-слой, отвечающий за авторизацию. Однако на макроуровне мы все еще имеем простую клиент- сервер архитектуру.

И вот наш прокси-сервер превращается просто в сервер в случае, когда к нему обращается клиент, и наоборот, выступает в роли клиента, когда обращается к серверу дальше.

То есть прокси действует как сервер для начального клиента, который посылает запрос, а затем как клиент, когда ретранслирует эту просьбу.





То есть прокси действует как сервер для начального клиента, который посылает запрос, а затем как клиент, когда ретранслирует эту просьбу.

Шлюз — это еще один дополнительный компонент, он переводит HTTP-запрос в другой протокол, распространяет этот запрос, а затем переводит полученный ответ обратно в HTTP. Клиент может обращаться со шлюзом, как с обычным сервером. Пример шлюза — система, которая загружает файлы с FTP-сервера.

Таким образом, несмотря на то, что у нас могут быть десятки слоев в системе, каждый компонент знает только о существовании ближайшего уровня к нему. Ограничивая знания о системе всего одним слоем, мы ограничиваем общую сложность системы.

6 Код по требованию

Код по требованию — единственное опциональное ограничение, которое предполагает отправку сервером исполняемого кода клиенту. Это то, что происходит в HTML-теге <script>.

Сервер в ответе может прислать клиенту код для непосредственного исполнения. Чаще всего это куски JavaScript кода, который без особых проблем исполняется в браузере.

Это то, что происходит в HTML-теге <script>. Когда HTML-документ загружается, браузер автоматически выбирает на сервере JavaScript и исполняет его локально.

Так общий код приложения на клиенте становится сложнее только при необходимости.

REpresentational State Transfer

Клиент-сервер
(Client-server)

Интерфейс является отдельной программой от сервера с бизнес логикой и базой данных.

Независим от состояния
(Stateless)

Сервер не хранит и не запоминает состояние клиента. Каждый запрос понятен в отрыве от контекста.

Кэшируемый
(Cache)

В сообщении может быть пометка, что данные можно закешировать и обращаться к ним из памяти.

Унификация интерфейса
(Uniform interface)

Идентификация ресурса
(identification of resources)

Каждый ресурс должен иметь уникальный идентификатор.

Манипуляция через представление
(manipulation through representations)

Клиент предоставляет серверу ресурс в уже измененном виде. А сервер уже решает применить эти изменения или нет.

Самоописанные сообщения
(self-descriptive messages)

Данных в сообщении достаточно, чтобы их понять и интерпретировать.

Гипермедиа как двигатель состояния
(hypermedia as the engine of application state)

Сервер подсказывает клиенту какие дальнейшие запросы он может делать.

Многоуровневость
(Layered)

Серверов может быть большое множество. Но каждый знает только о существовании ближайшего.

Код по запросу
(Code on demand)

Сервер может отправить на клиент код для непосредственного исполнения.

Принцип построения URI:

Для описания ресурса используй существительное во множественном числе.

```
/api/posts - коллекция ресурсов
/api/posts/42 - конкретный ресурс
```

URI может быть составным, если ресурс существует в контексте друго ресурса

```
/api/posts/42/comments - коллекция ресурсов
/api/posts/42/comments/12 - конкретный ресурс
```

Заголовки запроса:

Accept – типы медиа, разрешенные в ответе

Authorization – авторизационные данные для запроса

Cache-Control – определяет, разрешено ли кэширование

Content-Type – тип тела запроса

User-Agent – ПО, при помощи которого осуществлен запрос – например, браузер

Заголовки ответа:

Cache-Control – определяет, можно ли кэшировать запрос, и как надолго

Content-Encoding – тип кодировки в данных

Content-Type – формат ответа

Location – определяет, был ли запрос перенаправлен, или где создавался ресурс

Методы запроса:

GET - получить ресурс или коллекцию

POST - создание или запрос без сайд-эффекта

PUT - обновление

DELETE - удаление

HTTP статус коды:

1xx - Информационные
2xx - Успешные
3xx - Перенаправления
4xx - Ошибки запроса
5xx - Ошибки сервера

200 OK
Запрос успешно обработан

201 Created
Новый ресурс создан

301 Moved Permanently
URL ресурса изменился. Новый адрес в ответе.

400 Bad Request
Ошибка в запросе. Неверные данные.

401 Unauthorized
Клиент не авторизован для этого запроса

404 Not Found
Запрашиваемый ресурс не найден

500 Internal Server Error
Ошибка на сервере

Полный список всех кодов: ссылка

Протоколы

HTTP (HyperText Transfer Protocol, протокол передачи гипертекста).

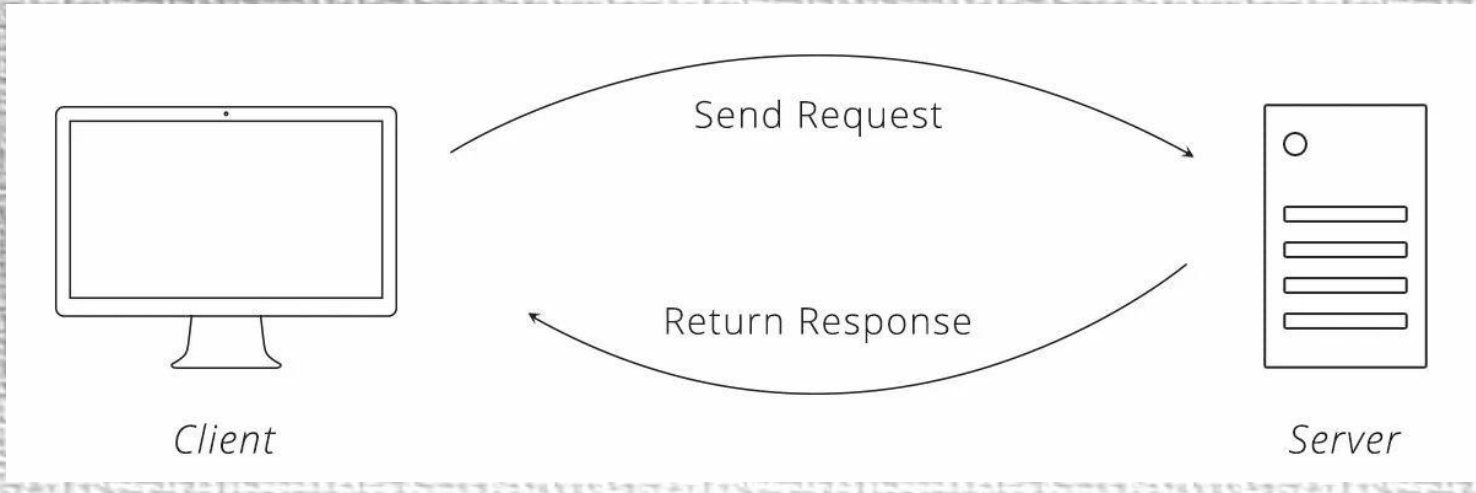
В интернете основным протоколом является протокол передачи гипертекста, более известный под аббревиатурой HTTP (HyperText Transfer Protocol, протокол передачи гипертекста). Когда вы вводите адрес вроде **`http://example.com`** в веб-браузере, «http» указывает браузеру использовать правила HTTP-протокола при разговоре с сервером.

С повсеместным распространением HTTP в интернете многие компании предпочитают адаптировать его для использования в качестве протокола, лежащего в основе своих API-интерфейсов. Одним из преимуществ использования этого протокола является то, что он прост в изучении разработчиками. Еще одним преимуществом является то, что HTTP имеет несколько свойств, полезных для создания хорошего API.

HTTP-запросы

Взаимодействие в HTTP основано на концепции, называемой Циклом «Запрос-Ответ». Клиент отправляет серверу запрос на выполнение каких-либо действий.

Сервер, в свою очередь, отправляет клиенту ответ, в котором говорит, может ли он сделать то, что просил клиент.



Чтобы сделать правильный запрос, клиент должен указать четыре вещи:

1. **URL** (Unified Resource Locator, единый указатель ресурсов)
2. **Метод** (Method)
3. **Список Заголовков** (Headers)
4. **Тело** (Body)

1. URL - унифицированный определитель местонахождения ресурса

В HTTP URL — это уникальный адрес какого-либо объекта. Каким объектам давать адреса — полностью решает организация, которая управляет сервером. Компании могут создавать URL-адреса для веб-страниц, изображений или даже видео с котиками.

API развивают эту идею, добавляя в контекст URL такие объекты, как, например, клиенты, продукты и твиты. В результате URL-адреса становятся для клиента простым способом сообщить серверу, с каким объектом он хочет взаимодействовать.

Конечно, API не называют их «объектами», им дают техническое название «ресурсы».

Термины URI, URL, URN

- **URI** - Uniform Resource Identifier (унифицирован ный идентификатор ресурса). Обозначает имя и адрес ресурса в сети. Как правило, делится на URL и URN.
- **URL** - Uniform Resource Locator (унифицированный определитель местонахождения ресурса). Адрес некоторого ресурса в веб. URL определяет местонахождение ресурса и способ обращения к нему.
- **URN** - Unifrorm Resource Name (унифицированное имя ресурса). Хороший пример это ISBN у книг (например, ISBN 978-5-699-12014-7 однозначно идентифицирует книгу, но ничего не говорит о ее местоположении). Смысл URN в том, что он определяет только название конкретного предмета, который может находиться во множестве конкретных мест.

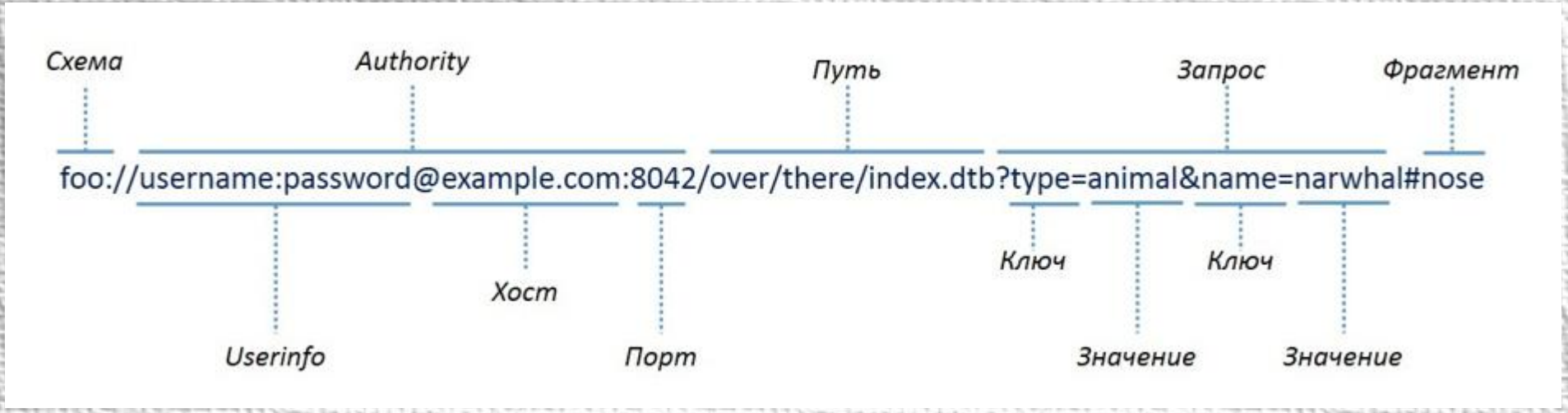
Можно сказать, что

URI = URL + URN

Приведу простой пример как это может выглядит на практике:

- URN - /products.html
- URL - https://mydomain.com
- URI - https://mydomain.com/products.html

Еще пример структуры URI в виде картинки:



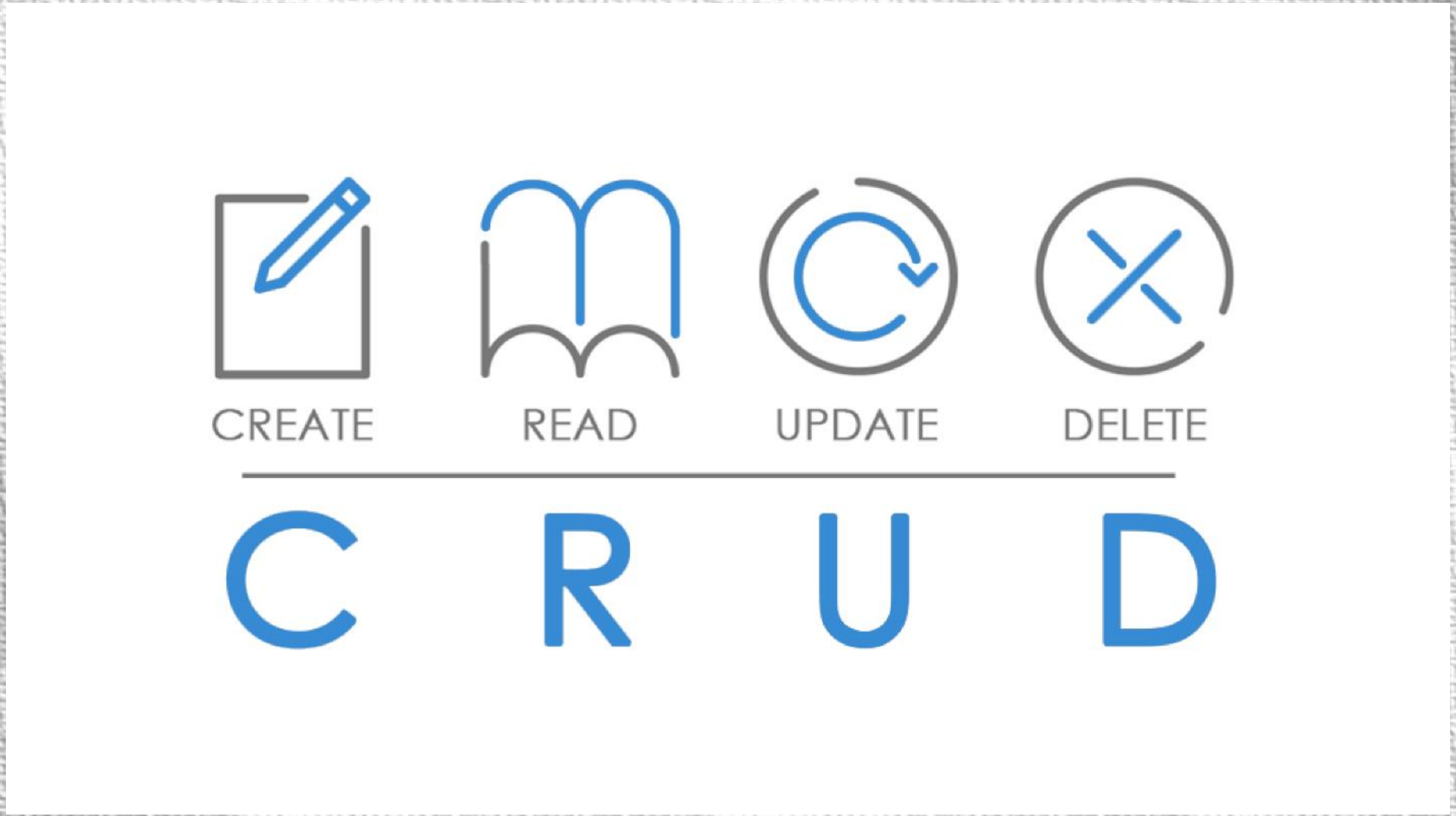
2. Метод (method)

Метод запроса сообщает серверу, какое действие клиент хочет, чтобы сервер предпринял. Фактически, метод обычно называют «глаголом» запроса.

В API-интерфейсах чаще всего встречаются четыре метода (вспомним **CRUD**):

1. **GET** — просит сервер получить ресурс
2. **POST** — просит сервер создать новый ресурс
3. **PUT** — просит сервер отредактировать/обновить существующий ресурс
4. **DELETE** — просит сервер удалить ресурс

МЕТОДЫ



Пример

Вот пример, который поможет проиллюстрировать эти методы. Допустим, есть пиццерия с API, который можно использовать для размещения заказов. Вы размещаете заказ, отправляя **POST-запрос** на сервер ресторана с перечнем вашего заказа с просьбой создать вашу пиццу по этим ингридиентам. Однако после того, как вы отправили запрос, вы понимаете, что выбрали неправильные ингредиенты, поэтому вы делаете запрос **PUT**, чтобы изменить его.

Ожидая своего заказа, вы делаете кучу запросов **GET** для проверки статуса. После часа ожидания вы решаете, что не готовы больше ждать, и делаете **DELETE-запрос**, чтобы отменить свой заказ.

3. Заголовки (Headers)

Заголовки предоставляют метаинформацию о запросе. Это простой список элементов, таких как время отправки запроса клиентом и размер тела запроса.

Помните, как вы заходили со своего смартфона на веб-сайт, специально отформатированный для мобильных устройств? Это стало возможным благодаря HTTP-заголовку под названием «User-Agent». Клиент использует этот заголовок, чтобы сообщить серверу, какой тип устройства вы используете, и если веб-сайт достаточно умён чтобы прочесть его, то в результате отправит вам данные в наиболее удобном для вашего типа устройства формате.

4. Тело (Body)

В теле запроса содержатся данные, которые клиент хочет отправить серверу. В примере с заказом пиццы выше, тело — это место, где размещаются ингредиенты заказа.

Уникальной чертой тела является то, что клиент полностью контролирует эту часть запроса. В отличие от метода, URL-адреса или заголовков, где протокол HTTP требует жёсткой структуры, тело позволяет клиенту отправлять всё, что ему нужно.

Эти четыре части — URL, метод, заголовки и тело — составляют полный HTTP-запрос.

HTTP-ответы
(HTTP-response)

Ответ, как правило, состоит из следующих частей:

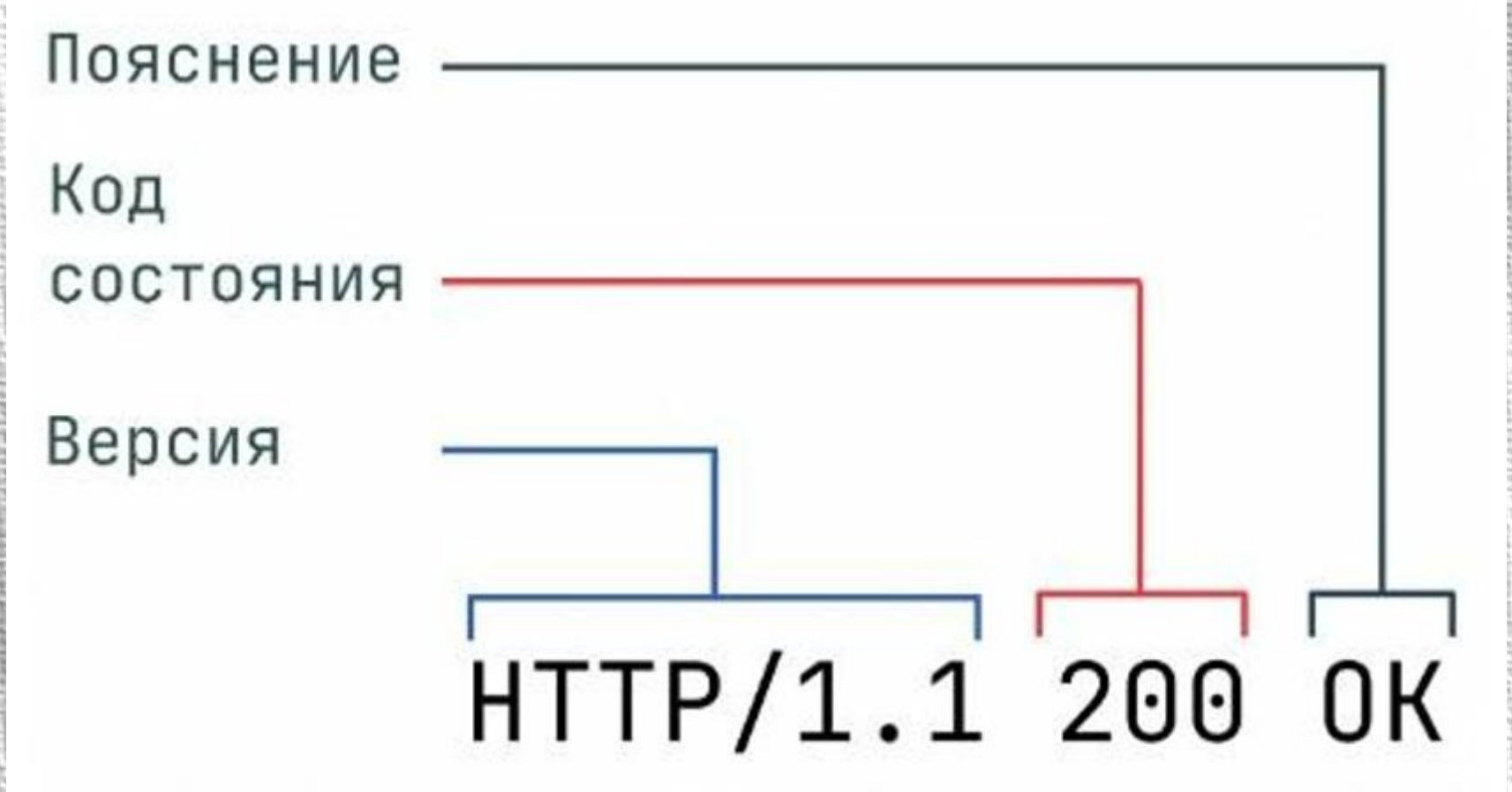
- **код ответа;**
- **заголовки;**
- **тело ответа.**

В целом заголовки ответов мало чем отличаются от заголовков запросов. К тому же некоторые заголовки используются и в ответах, и в запросах. С телом ответа тоже, думаю, все понятно. В теле часто возвращается информация, которую запросил клиент. Может возвращаться в том же формате JSON информация на GET запросы. А вот последняя часть, немного более интересна.

Стартовая строка HTTP-ответа называется строкой статуса (status line). На ней располагаются следующие элементы:

- **Версия протокола** (HTTP/2 или HTTP/1.1).
- **Код состояния**, который указывает, насколько успешно завершилась обработка запроса.
- **Пояснение** — короткое текстовое описание к коду состояния. Используется исключительно для того, чтобы упростить понимание и восприятие человека при просмотре ответа.

Строка состояния ответа



Коды HTTP ответов

Рассмотрим подробнее коды HTTP ответов. Приведем цитату из Википедии: Код состояния HTTP (англ. HTTP status code) — часть первой строки ответа сервера при запросах по протоколу HTTP. Он представляет собой целое число из трёх десятичных цифр. Первая цифра указывает на класс состояния. За кодом ответа обычно следует отделенная пробелом поясняющая фраза на английском языке, которая разъясняет человеку причину именно такого ответа. Примеры:

- **201 Created;**
- **401 Unauthorized;**
- **507 Insufficient Storage.**

Клиент узнаёт по коду ответа о результатах его запроса и определяет, какие действия ему предпринимать дальше.

Форматы данных

1. JSON (JavaScript Object Notation)
2. XML (Extensible Markup Language)

Что такое представление?

Пусть сервер должен вернуть такой ресурс как **клиент**.

- Это будет привычный текстовый формат с ФИО, адресом, почтой и т.п.?
- А может быть это будет его фотография?
- А может быть видео о нем?
- А может быть…?

Это все и есть возможное формы представления ресурса.

При обмене данными с людьми возможности отображения информации ограничиваются только человеческой фантазией. Например, как пиццерия может оформить своё меню? Это может быть текстовый список или маркированный список, может быть и серией фотографий с подписями или только набором фотографий, на которые показывают посетители, чтобы оформить заказ.

Хорошо продуманный формат помогает целевой аудитории понимать информацию.

Тот же принцип применяется при обмене данными между компьютерами. Один компьютер должен поместить данные в формат, понятный для другого. Обычно это какой-то текстовый формат.

Формат — это способ описания данных

С ним можно работать двумя способами:

1. **упаковать данные** — то есть сериализовать их
2. **извлечь данные** — десериализовать их

Задача сериализации и десериализации возникает тогда, когда нам нужно передать данные из программы наружу — например, другим программам.

Данные внутри языков представляются каким-то способом, специфичным для данного языка и даже его конкретной версии. Поэтому для передачи данных и используются универсальные форматы, которые известны всем.

В случае HTTP API этот механизм работает так:

- Сервис, который предоставляет HTTP API, извлекает данные из хранилища, формирует JSON и отдает его наружу
- Затем этот JSON может прочитать любая программа с поддержкой JSON (а это большинство современных программ)

Поддержка JSON часто реализована прямо на уровне языков программирования. JSON — это всего лишь текст. У него есть понятная структура, которая прослеживается визуально. Отступы, пробелы и переносы для JSON не имеют значения.

JSON — это не единственный формат данных. До него популярным форматом был XML, и сейчас он встречается довольно часто. XML похож на HTML, но решает другую задачу. XML — это формат данных, как и JSON. Разница лишь в том, что XML не предназначен для вывода.

Доступные форматы данных

API REST могут использовать любой формат сообщений, который хотят использовать разработчики API, включая XML, JSON, Atom, RSS, CSV, HTML и другие.

1. JSON (JavaScript Object Notation)
2. XML (Extensible Markup Language)

Чем это отличается от передачи данных в HTML

HTML — обычно служит для передачи данных в браузеры, так как это язык разметки, с помощью которого формируется текст для браузеров. Браузеры считывают HTML и отображают его в виде веб-страницы. HTML не подразумевает работу с данными, которые содержатся внутри него. Теоретически это можно сделать, но на практике будет очень сложно.

Как форматы данных используются в HTTP

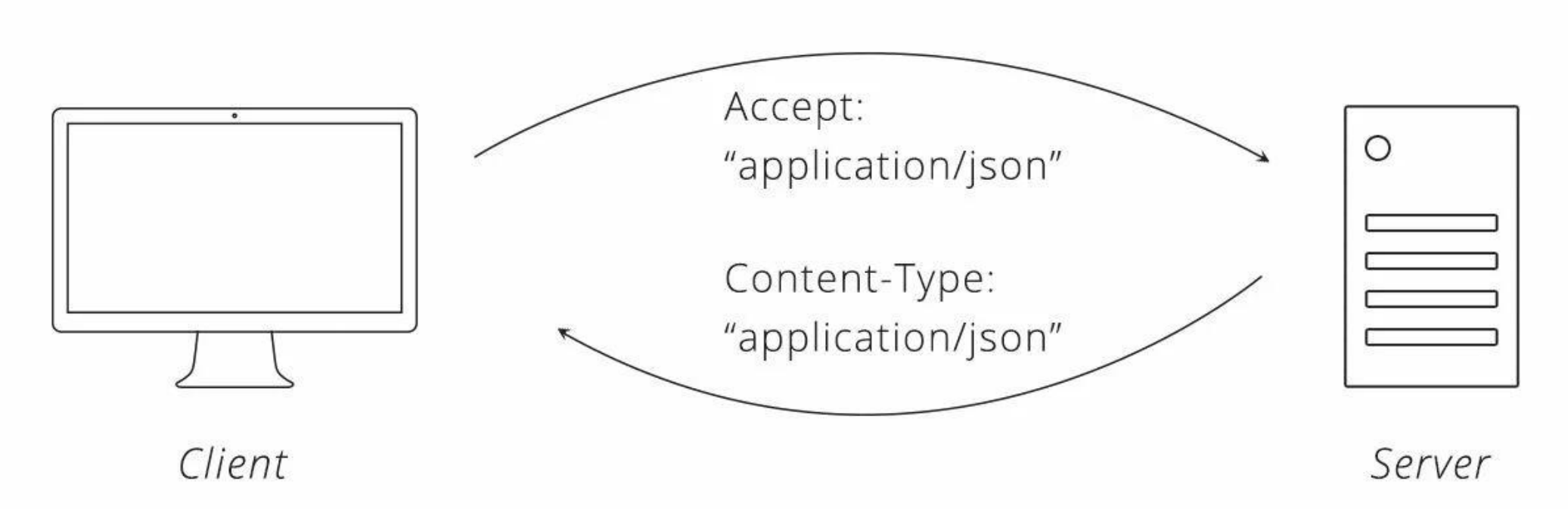
Теперь, когда мы изучили некоторые доступные форматы данных, нам нужно знать, как использовать их в HTTP. Вспомним о заголовках. Есть заголовок, указывающий, в каком формате находятся данные: **`Content-Type`** Когда клиент отправляет заголовок **`Content-Type`** в запросе, он сообщает серверу, что данные в теле запроса отформатированы определённым способом. Если клиент хочет отправить серверу данные в формате JSON, он установит **`Content-Type`** на «`application/json`». Получив запрос и увидев этот **`Content-Type`**, сервер сначала проверит, понимает ли он этот формат, и, если да, он будет знать, как читать данные. Аналогичным образом, когда сервер отправляет клиенту ответ, он также устанавливает **`Content-Type`**, чтобы сообщить клиенту, как читать тело ответа.

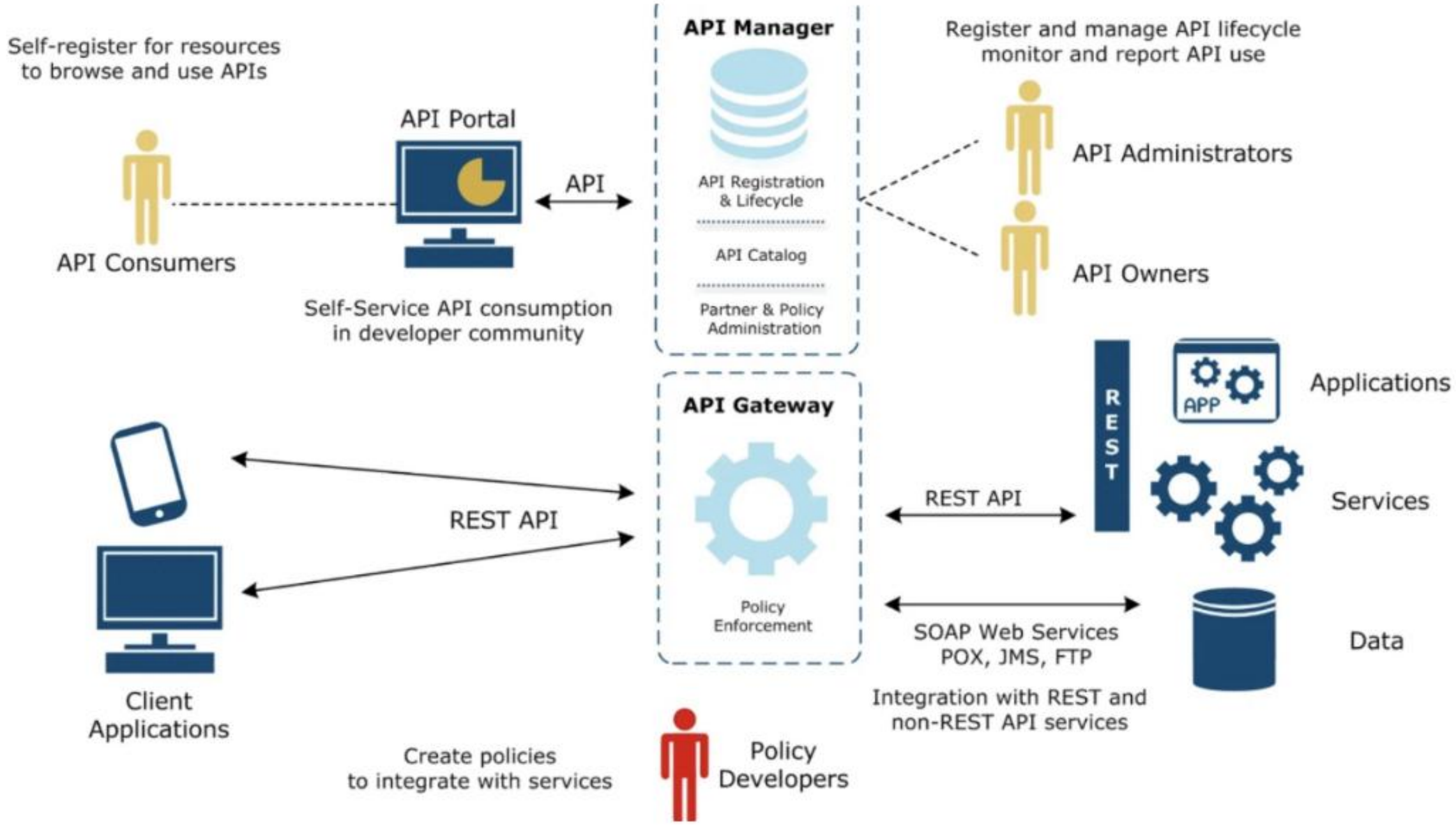
Иногда клиент может общаться только в одном формате данных. Если сервер отправляет обратно что-либо, кроме этого формата, клиент откажется и выдаст ошибку. К счастью, на помощь приходит второй HTTP-заголовок. Клиент может установить заголовок **`Accept`**, чтобы сообщить серверу, какие форматы данных он может принимать. Если клиент может общаться только в формате JSON, он может установить для заголовка **`Accept`** значение «application/json». И тогда сервер будет отправлять ответы в формате JSON. Если сервер не поддерживает формат, запрашиваемый клиентом, он может отправить клиенту сообщение об ошибке, чтобы сообщить ему, что запрос не будет работать. Это можно интерпретировать примерно так: «Я посылаю сообщение на русском языке, но могу общаться и на английском. Или только на английском»

С помощью этих двух заголовков, **`Content-Type`** и **`Accept`**, клиент и сервер могут работать с форматами данных, которые они понимают и должны работать должным образом.

Заголовки форматов данных

Content-Type и Accept




Self-register for resources
to browse and use APIs
API Portal
API
API Consumers
Self-Service API consumption
in developer community
API Manager
API Registration
& Lifecycle
API Catalog
Partner & Policy
Administration
Register and manage API lifecycle
monitor and report API use
API Administrators
API Owners
API Gateway
Policy
Enforcement
REST API
REST API
REST
APP
Applications
Services
SOAP Web Services
POX, JMS, FTP
Data
Integration with REST and
non-REST API services
Client
Applications
Create policies
to integrate with services
Policy
Developers



Open Data Protocol

OData (Open Data Protocol) — это стандарт, утверждённый ISO/IEC и OASIS, который определяет набор лучших практик для создания и потребления REST API.

Он позволяет создавать REST-сервисы, которые позволяют ресурсам, идентифицированным с помощью Uniform Resource Locators (URL) и определённым в модели данных, публиковаться и редактироваться веб-клиентами с помощью простых HTTP-сообщений.

Протокол позволяет выполнять операции с ресурсами, используя в качестве запросов HTTP-команды, и получать ответы в форматах XML или JSON.

Концепции OData

В основе протокола OData лежит концепция Entity Data Model, или сокращенно EDM. EDM описывает данные, предоставляемые поставщиком OData через документ метаданных, содержащий ряд метасущностей:

- Тип объекта и его свойства (например , Person , Customer , Order и т. д.) и ключи
- Отношения между сущностями
- Сложные типы, используемые для описания структурированных типов, встроенных в сущности (например, тип адреса, являющийся частью типа Customer) .
- Наборы сущностей, объединяющие сущности данного типа.

Спецификация требует, чтобы этот документ метаданных был доступен в стандартном расположении **$metadata** по корневому URL-адресу, используемому для доступа к службе. Например, если у нас есть служба OData, доступная по адресу **http://example.org/odata.svc/** , то ее документ метаданных будет доступен по адресу **http://example.org/odata.svc/$metadata** .

Формат запросов в REST API

Запросы строятся в формате OData, подробнее можно посмотреть по адресу http://docs.oasis-open.org/odata/odata/v4.0/odata-v4.0-part2-url-conventions.html

1. Метаданные
2. Пагинация
3. Фильтрации данных
4. Сортировка
5. Выборка данных

Типы параметров в API

Наиболее распространённые типы параметров, используемых в API:

- **Параметры пути**. Параметры в пределах path конечной точки перед строкой запроса, отделяются знаком ?. Обычно эти параметры выделяются фигурными скобками.
- **Параметры строки запроса**. Их можно указать в строке запроса конечной точки. Располагаются после знака ?.
- **Параметры заголовка**. Включены в заголовок запроса, обычно относятся к авторизации.
- **Параметры тела запроса**. Параметры, включенные в тело запроса. Обычно в формате JSON.

В REST API можно организовать параметры как минимум двумя способами:

1. **К**ак часть URL-пути через аргументы:
   `/api/resource/parametervalue`
2. **В** качестве аргумента запроса, query parameters:
   `/api/resource?parameter=value`

Первый пример `/api/resource/parametervalue` используется когда вам параметр нужен как часть пути или привязки к чему-то.

Второй пример содержит query parameters, т.е. те, которые идут после амперсанда. Это значит, это они не обязательные. К примеру, вы можете использовать их как часть фильтров для get запросов. Для post запросов они в основном не используются потому что вы и так передаёте данные, но уже в теле запроса.

Параметры запроса и параметры маршрутизации это разные вещи, которые делают примерно одно и то самое - передают данные.

Параметры маршрутизации используются для уточнения пути к объекту, с которым вы собираетесь работать, а параметры запроса используются больше как фильтры при некотором запросе и являются не обязательными.

Параметры запроса в API

Параметры запроса в API — это опции, которые можно передать с конечной точкой, чтобы повлиять на ответ. Они похожи на поисковые фильтры: они выделяют данные, которые нужно получить от API.

Параметры строки запроса указываются после знака вопроса **?** в конечной точке. Знак вопроса, параметры, которые следуют за ним и их значения, называется «строкой запроса».

В строке запроса каждый параметр перечисляется один за другим с амперсандом **&**, разделяющим их. Порядок параметров строки запроса не имеет значения.

Параметры запроса

- Параметр запроса **$filter**
- Параметр запроса **$expand**
- Параметр запроса **$select**
- Параметр запроса **$orderby**
- Параметры запроса **$top и $skip**
- Вариант запроса **$count**
- Параметр запроса **$search**
- Параметр запроса **$format**

Типы данных параметров

API могут некорректно обрабатывать параметр, если он имеет неправильный тип данных или неправильный формат. Перечисление типа данных является хорошей идеей для всех типов параметров, но особенно важно для параметров тела запроса, поскольку они обычно форматируются в JSON.

Вот типы данных наиболее распространенные в REST API:

- **string:** буквенно-цифровая последовательность букв и / или цифр;
- **integer**: целое число - может быть положительным или отрицательным;
- **boolean**: true или false значение;
- **object**: пара ключ-значение в формате JSON
- **array**: массив значений

Максимальное и минимальное значения параметров

Помимо указания типа данных, параметры должны указывать максимальные, минимальные и допустимые значения. Например, если API сервиса погоды допускает только координаты долготы и широты конкретных стран, эти ограничения должны быть описаны в документации в разделе параметров.

Пропуск информации о максимальных / минимальных значениях или других недопустимых значениях является распространенной ошибкой в документации. Разработчики часто не осознают всех «творческих» способов, которыми пользователи могут использовать API. Команда тестирования или обеспечения качества (QA), вероятно, является лучшим ресурсом для определения значений, которые не должны допускаться, потому что задача QA - попытаться взломать API.

Пагинация

Функции, возвращающие массив значений, поддерживают параметры для порционной загрузки, которые передаются в виде *query-строки* (`?$top=1&$skip=10`).

`https://localhost:9000/api/v1/Docs?$top=5&$skip=10`

При выполнении данного запроса мы получим максимум 5 записей начиная с 11 индекса в списке.

Если в запросе на получение списка указать параметр count=true, в результате вернется дополнительно поле count, которое содержит общее число записей в списке на сервере.

Фильтрации данных

Для фильтрации данных в запросе можно указывать параметр **$filter**, например:

```
https://localhost:9000/api/v1/Docs?$filter=lastChangeDate gt
2017-10-06T17:41:10Z and documentTypeName eq ‘Заказ'
```

При выполнении данного запросы мы получим список документов, дата изменения которых более указанной в запросе **и** тип документа — «Заказ».

Сортировка

Чтобы отсортировать результат необходимо указать параметр **`$orderby`**, например:

`https://localhost:9005/api/v1/Docs?$orderby=id`

В результате получим список документов, отсортированный по идентификатору.

`https://dummyjson.com/users?$orderby=age`

Выборка данных

Представим, что нам нужны не все данные, а только их часть. Для этого в нашем HTTP API есть параметр запроса `select`:

`https://dummyjson.com/users?select=firstName,gender`.

`firstName,gender` – список полей, которые необходимо отобразить в результате.

Так выглядит результат:


```
{
    "users": [
        {
            "id": 1,
            "firstName": "Terry",
            "gender": "male"
        },
        {
            "id": 2,
            "firstName": "Sheldon",
            "gender": "male"
        },
        {
            "id": 3,
            "firstName": "Terrill",
            "gender": "male"
        }
    ]
}
```

Одиночный ресурс
(Выборка записи по идентификатору)

Эндпоинт `/users` возвращает список пользователей. Если нам нужен один пользователь, то для этого понадобится другой эндпоинт — `/users/{id}`.

Этот эндпоинт называется динамическим, потому что у него есть меняющаяся часть. Вместо {`id`} подставляется идентификатор конкретного пользователя, данные которого мы хотим получить.

Возьмем для примера `https://dummyjson.com/users/9` и посмотрим на результат:

Вложенные ресурсы

Часто пользователи могут писать посты на сайте. Если мы захотим увидеть весь список постов, то используем эндпоинт `/posts`.

Чтобы увидеть посты конкретного пользователя, мы воспользуемся вложенными ресурсами: `https://dummyjson.com/users/3/posts`.

Этот *эндпоинт* вернет все посты пользователя с идентификатором 3:

По такому же принципу устроена работа со всеми остальными ресурсами:

URL	Описание
/users/{id}/posts	Список постов пользователя
/users/{id}/comments	Список комментариев пользователя
/users/{id}/todos	Список задач пользователя

Документирование параметров тела сложного запроса

Документирование данных JSON (как в параметрах тела запроса, так и в ответах) является одной из самых сложных частей документации API. Документирование JSON объекта будет легким, если этот объект прост, с несколькими парами ключ-значение на одном уровне. Но если у нас есть JSON объект с несколькими объектами внутри объектов, множественными уровнями вложенности и большими объемными данными, это может быть сложно. И если объект JSON занимает более 100 строк, или 1000, нам необходимо тщательно продумать, как представить информацию.

Таблицы хороши для документирования JSON, но в них трудно различать элементы верхнего уровня и подуровня. Объект, который содержит объект, который также содержит объект, и другой объект и т. Д., Может сбивать с толку.

Безусловно, если объект JSON относительно мал, таблица, вероятно, является лучшим вариантом. Но есть и другие дизайнерские подходы.

Подход Swagger к параметрам тела запроса

Пользовательский интерфейс Swagger, который мы рассмотрим позже, а также его демо, предоставляет другой подход к документированию параметра тела запроса. Swagger UI показывает параметры тела запроса в формате, который вы видите ниже. Интерфейс Swagger позволяет переключаться между представлением «Пример значения» и представлением «Модель» для ответов и параметров тела запроса.

Операторы сравнения OData в поиске ИИ Azure

Операторы равенства:

- **eq:** проверяет, является ли поле равным постоянному значению.
- **ne**: проверяет, является ли поле не равным постоянному значению.

Операторы диапазона:

- **gt**: проверяет, больше ли поле постоянного значения.
- **lt**: проверяет, меньше ли поле постоянного значения.
- **ge**: проверяет, больше или равно поле постоянному значению.
- **le**: проверяет, меньше или равно поле постоянному значению.

Логические операторы OData в поиске ИИ Azure

- **and**. Бинарный оператор, возвращающий значение `true`, если результат вычисления левого и правого операндов имеет значение `true`.
- **or**. Бинарный оператор, возвращающий значение `true`, если результат вычисления левого или правого операнда имеет значение `true`.
- **not**. Унарный оператор, возвращающий значение `true`, если его операнд имеет значение `false`, и наоборот.

Аутентификация

1. Обычная проверка подлинности
2. Аутентификация с помощью ключа API
3. Открытая авторизация

Аутентификация

Как сервер узнает, что клиент является тем, кем он себя называет? Ниже мы исследуем два способа, с помощью которых клиент может подтвердить свою личность серверу.

Вам, вероятно, уже много раз приходилось регистрировать учетную запись на каких-либо веб-сайтах. В ходе этого процесса сайт запрашивает у вас некоторую личную информацию, в первую очередь имя пользователя и пароль. Эти две части информации становятся вашими опознавательными знаками. Мы называем их вашими полномочиями. При повторном посещении веб-сайта вы можете войти в него, указав эти учетные данные.

Вход в систему с использованием имени пользователя и пароля — один из примеров технического процесса, известного как **аутентификация**. Когда вы аутентифицируетесь на сервере, вы подтверждаете свою личность серверу, сообщая ему информацию, которую знаете только вы (по крайней мере, мы надеемся, что это знаете только вы). Как только сервер узнает, кто вы, он сможет вам доверять и предоставить личные данные вашей учетной записи.

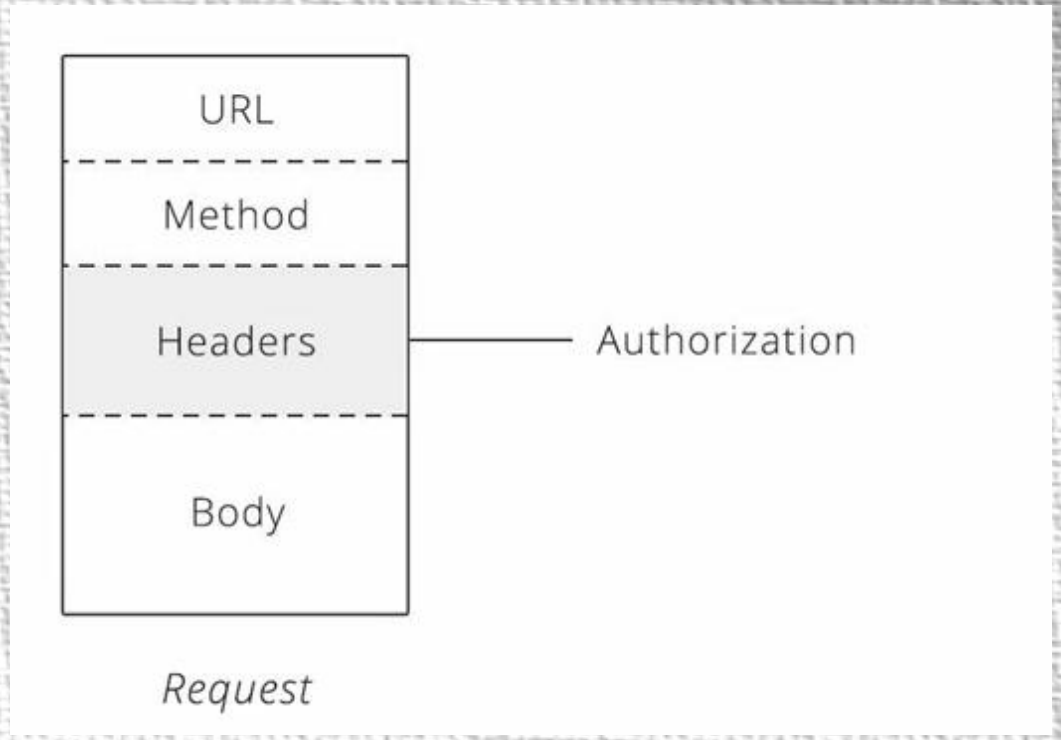
API-интерфейсы используют несколько методов для аутентификации клиента. Это так называемые *схемы аутентификации*. Давайте теперь посмотрим на две из этих схем.

❶ Обычная проверка подлинности

Приведенный выше пример входа в систему является самой простой формой аутентификации. Фактически, его официальное название — ***Базовая аутентификация*** («Базовая аутентификация» для его друзей). Хотя это имя и не получило никаких наград за творческие достижения, эта схема является вполне приемлемым способом для сервера аутентифицировать клиента в API.

Для базовой аутентификации требуется только имя пользователя и пароль. Клиент берет эти два учетных данных, смешивает их вместе, чтобы сформировать единое значение, и передает его в запросе в заголовке HTTP под названием «**Авторизация**» (**Authorization**).

Когда сервер получает запрос, он просматривает заголовок **Авторизации** и сравнивает его с сохраненными учетными данными. Если имя пользователя и пароль совпадают с одним из пользователей в списке сервера, сервер выполняет запрос клиента от лица этот пользователя. Если совпадений нет, сервер возвращает специальный код состояния (**401**), чтобы клиент знал, что аутентификация не удалась и запрос отклонен.



Хотя базовая аутентификация является допустимой схемой аутентификации, то обстоятельство, что она использует одно и то же имя пользователя и пароль как для доступа к API так и для управления учетной записью, вызывает опасения. Это все равно что отель бы передавал гостю ключи от всего здания, а не от комнаты.

Точно также с API могут быть случаи, когда клиент должен иметь другие разрешения, чем владелец учетной записи. Возьмем, к примеру, владельца бизнеса, который нанимает подрядчика для написания программы, использующей API от их имени. Передача подрядчику учетных данных подвергает владельца риску, поскольку недобросовестный подрядчик может изменить пароль, заблокировав владельцу бизнеса доступ к его собственной учетной записи. Понятно, что неплохо было бы иметь альтернативу.

Ключ API — это уникальная строка, состоящая из случайно сгенерированных чисел и букв, которая передается при каждом запросе в службу поиска. Служба примет запрос, если и сам запрос, и ключ действительны.

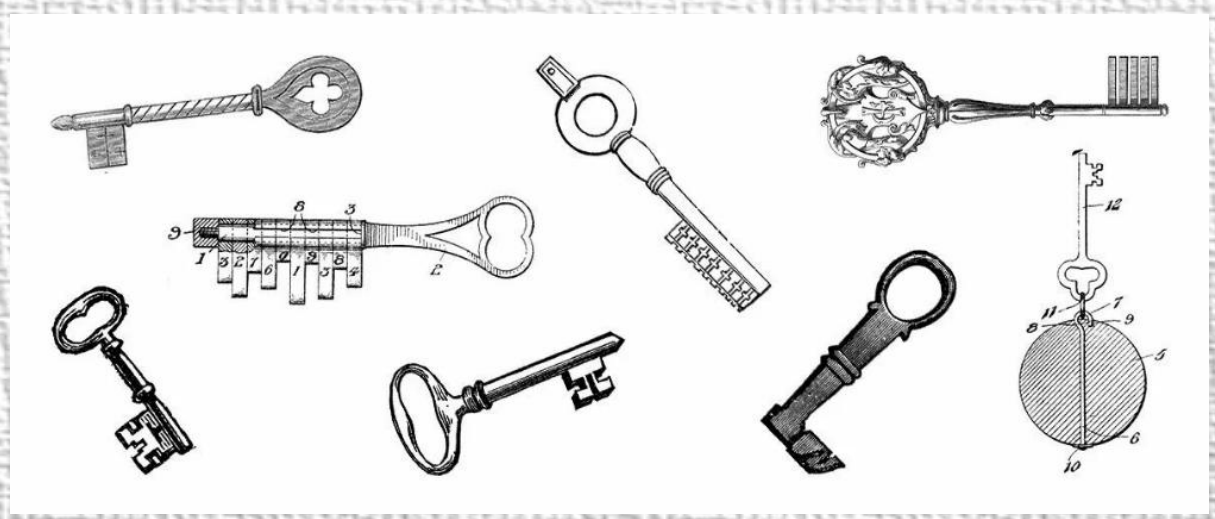
❷ Аутентификация с помощью ключа API

Аутентификация с помощью ключа API — это метод, который преодолевает слабость использования общих учетных данных, требуя доступа к API с помощью уникального ключа. В этой схеме ключ обычно представляет собой длинную последовательность букв и цифр, отличную от пароля для входа владельца учетной записи. Владелец дает ключ клиенту, так же как отель дает гостю ключ от одноместного номера.

Когда клиент аутентифицируется с помощью ключа API, сервер понимает, что можно разрешить клиенту доступ к данным, но теперь имеет возможность ограничить административные функции, такие как изменение паролей или удаление учетных записей.

Иногда ключи используются просто, чтобы пользователю не приходилось выдавать свой пароль. Гибкость обеспечивается аутентификацией API Key для ограничения контроля, а также защиты паролей пользователей.

Итак, где же находится ключ API? Заголовок для него тоже есть? К сожалению, нет. В отличие от **`Basic Auth`**, который является установленным стандартом со строгими правилами, ключи API были разработаны в нескольких компаниях на заре интернета. В результате аутентификация с помощью ключа API немного похожа на дикий запад; у каждого свой способ делать это.



Однако со временем появилось несколько общих подходов. Один из них заключается в том, чтобы клиент поместил ключ в заголовок **Авторизации** вместо имени пользователя и пароля. Другой — добавить ключ в URL-адрес (`http://example.com?api_key=my_secret_key`).

Реже ключ закапывают где-нибудь в теле запроса рядом с данными. Куда бы ни пошел ключ, эффект будет одинаковым — он позволяет серверу аутентифицировать клиента.

❸ Открытая авторизация
(Open Authorization, OAuth)

Все, кому когда-нибудь приходилось заполнять регистрационную форму с вводом длинного ключа в поле формы помнят свои неудобства. Во-первых, вам нужно найти требуемый ключ, который возможно вы уже и не помните. А если и помните, то неточно. Во-вторых, опечатка или пропуск одного символа приведет к сбою или даже может заблокировать вашу незарегистрированную программу.

Принуждение пользователей к работе с ключами API — тоже плохой опыт. Опечатки — обычная проблема, и она требует, чтобы пользователь вручную выполнял часть настройки между клиентом и сервером.

И тут появляется OAuth. Автоматизация обмена ключами — одна из основных проблем, которую решает OAuth. Он предоставляет клиенту стандартный способ получить ключ от сервера, проведя пользователя через простой набор шагов. С точки зрения пользователя, все, что требуется для OAuth — это ввод учетных данных. За кулисами клиент и сервер общаются туда-сюда, чтобы добиться работающего ключа для клиента.

В настоящее время существует две версии OAuth, метко названные OAuth 1 и OAuth 2 . Понимание шагов в каждом из них необходимо, чтобы иметь возможность взаимодействовать с API, которые используют их для аутентификации. Поскольку у них общий сценарий, мы пройдемся по этапам OAuth 2, а затем укажем, чем отличается OAuth 1.

OAuth 2

Вот набор персонажей, участвующих в обмене OAuth:

- **Пользователь** — человек, который хочет связать два веб-сайта.
- **Клиент** — веб-сайт, которому будет предоставлен доступ к данным пользователя.
- **Сервер** — веб-сайт, на котором хранятся данные пользователя.

Одна из целей OAuth 2 — позволить организациям адаптировать процесс аутентификации к своим потребностям. Из-за такой расширяемой природы у API могут быть несколько разные шаги. Показанный ниже сценарий является обычным для веб-приложений. В мобильных и настольных приложениях этот процесс может немного отличаться.

Итак, вот шаги OAuth 2.

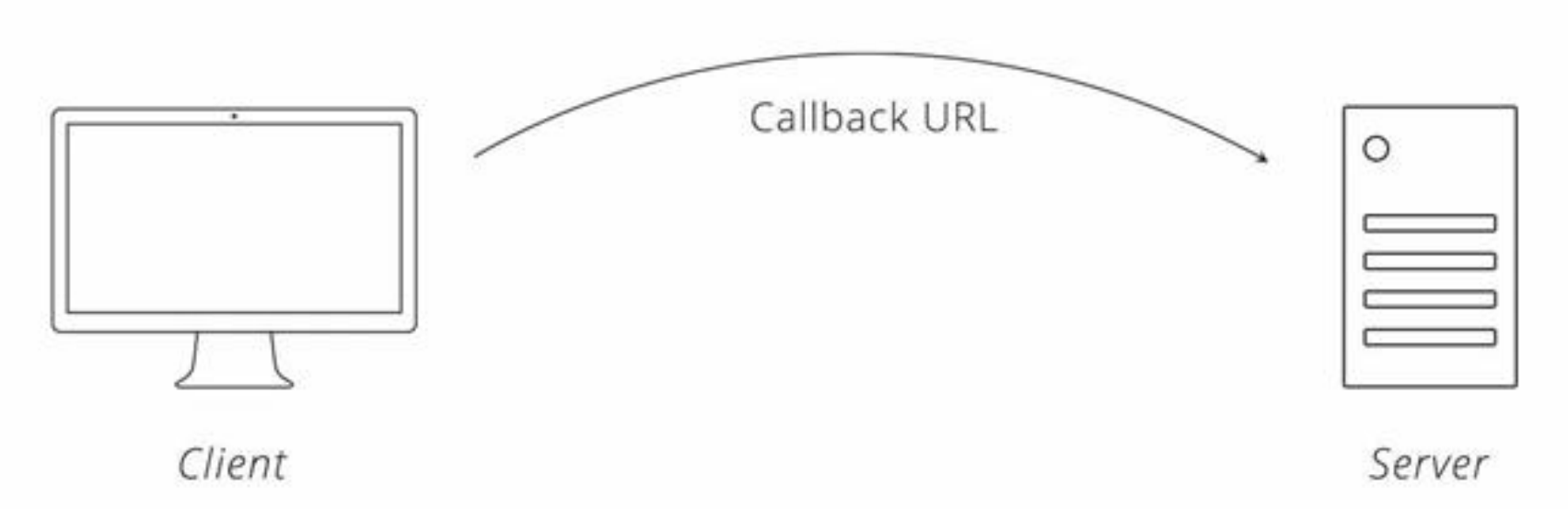
Шаг 1 — Пользователь сообщает клиенту о необходимости подключения к серверу



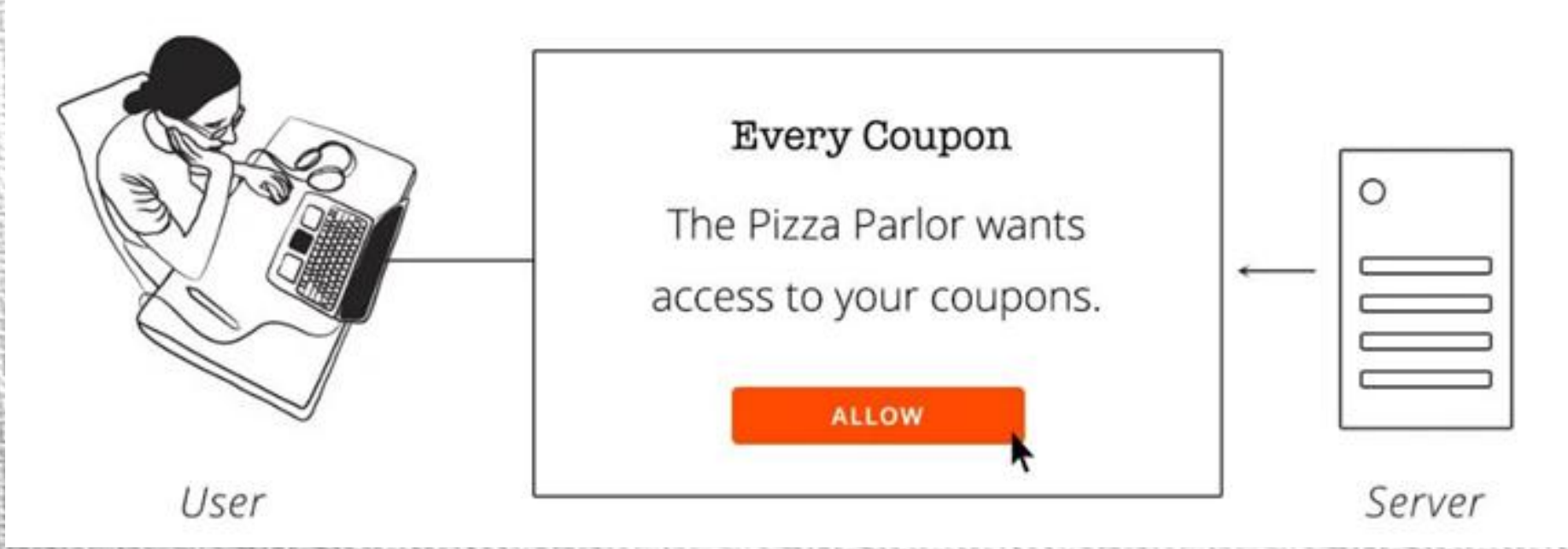
Пользователь запускает процесс, давая понять клиенту, что он хочет, чтобы он подключился к серверу. Обычно это делается нажатием кнопки.

Шаг 2 — Клиент направляет пользователя на сервер

Клиент отправляет пользователя на веб-сайт сервера вместе с URL-адресом, на который сервер отправит пользователя обратно после аутентификации пользователя, называемого URL-адресом обратного вызова (callback URL).

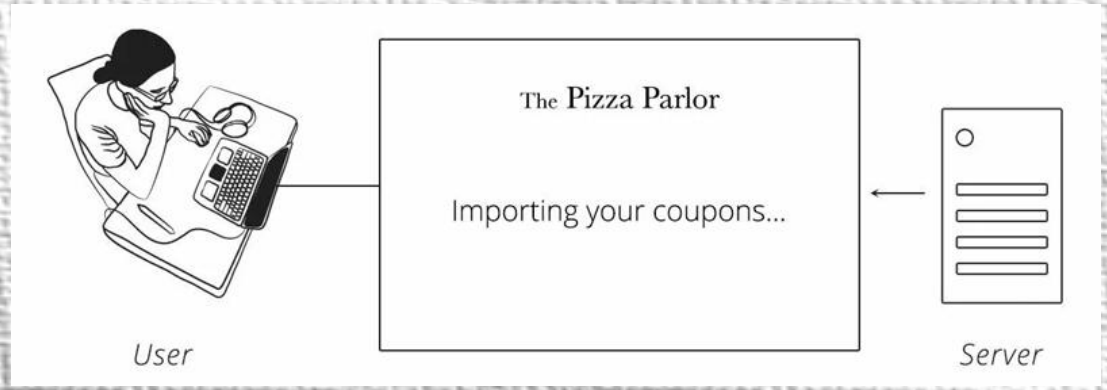


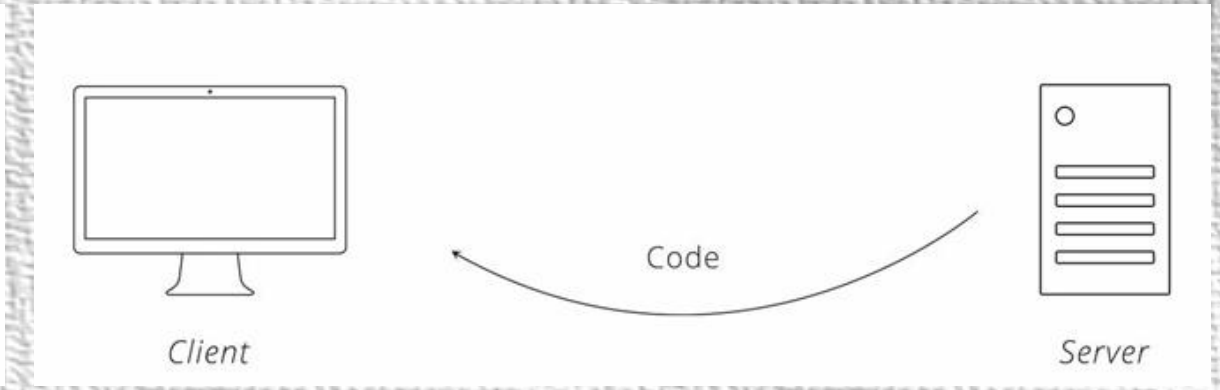
Шаг 3 — Пользователь входит на сервер и получает клиентский доступ



Используя свое обычное имя пользователя и пароль, пользователь аутентифицируется на сервере. Теперь сервер уверен, что один из его собственных пользователей запрашивает у клиента доступ к учетной записи пользователя и связанным данным.

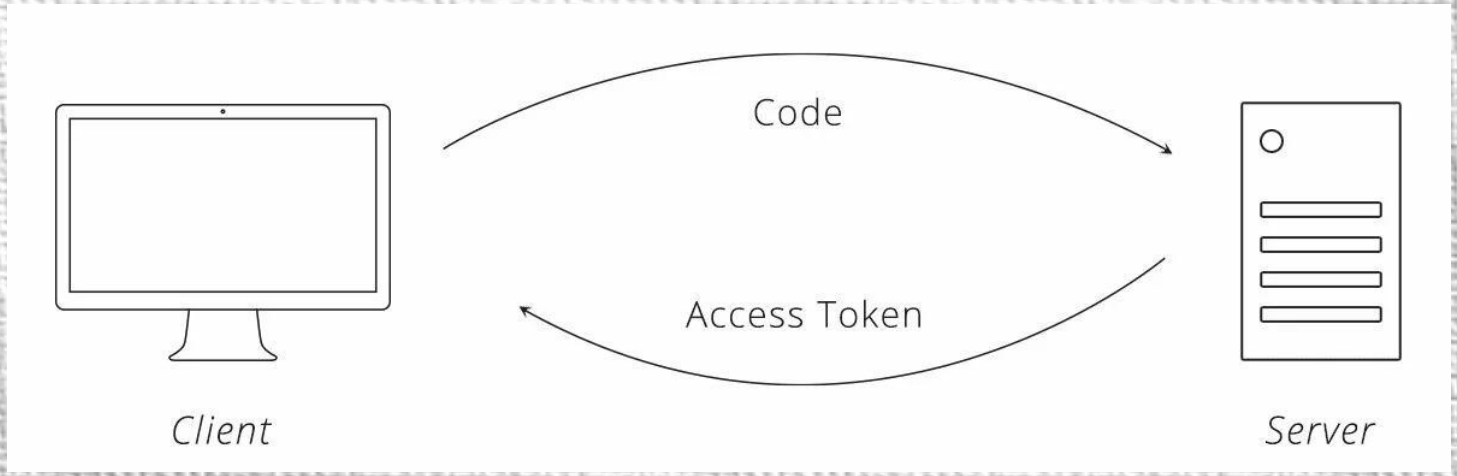
Шаг 4 — Сервер отправляет пользователя обратно клиенту вместе с кодом





Сервер отправляет пользователя обратно клиенту (на URL-адрес обратного вызова из шага 2). В ответе скрыт уникальный код авторизации для клиента.

Шаг 5 — Клиент обменивает Код + секретный Ключ на Токен доступа



Клиент берет полученный код авторизации и делает еще один запрос к серверу. Этот запрос включает секретный ключ клиента. Когда сервер видит действительный код авторизации и секретный ключ доверенного клиента, он уверен, что клиент является тем, кем он себя называет, и что он действует от имени реального пользователя. Сервер отвечает токеном доступа.

Шаг 6 — Клиент получает данные с сервера



На этом этапе клиент может получить доступ к серверу от имени пользователя. Маркер доступа из шага 6 — это, по сути, еще один пароль для учетной записи пользователя на сервере. Клиент включает токен доступа в каждый запрос, чтобы он мог аутентифицироваться напрямую с сервером.

Клиент обновляет токен (необязательно)

Функция, представленная в OAuth 2 — это возможность истечения срока действия токенов доступа. Это полезно для защиты учетных записей пользователей за счет усиления безопасности — чем быстрее истекает срок действия токена, тем меньше времени может быть злонамеренно использован украденный токен, подобно тому, как срок действия номера кредитной карты истекает через определенное время. Срок службы токена устанавливается сервером. API на практике могут использовать срок от часов до месяцев. По истечении срока службы клиент должен запросить у сервера новый токен.

Чем отличается OAuth 1

Между версиями OAuth существует несколько ключевых различий. Один мы уже упоминали; токены доступа не имеют срока действия.

Еще одно отличие состоит в том, что OAuth 1 включает дополнительный шаг. Между шагами 1 и 2, описанными выше, OAuth 1 требует, чтобы клиент запросил у сервера токен запроса. Этот токен действует как код авторизации в OAuth 2 и обменивается на токен доступа.

Третье отличие состоит в том, что OAuth 1 требует, чтобы запросы были подписаны цифровой подписью. Мы опустим детали того, как именно работает подписание (вы можете найти библиотеки кода, которые сделают это за вас), но стоит знать, почему оно есть в одной версии OAuth, но не в другой. Подписание запроса — это способ защитить данные от подделки при их передаче между клиентом и сервером. Подписи позволяют серверу проверять подлинность запросов.

Однако сегодня большая часть API-трафика происходит по уже защищенному каналу (HTTPS). Понимая это, OAuth 2 устраняет подписи, стараясь упростить использование второй версии. Торговля заключается в том, что OAuth 2 полагается на другие меры для обеспечения безопасности передаваемых данных.

Авторизация

Элементом OAuth 2, заслуживающим особого внимания, является концепция ограничения доступа, официально известная как авторизация. Вернемся к шагу 2, когда пользователь нажимает кнопку, чтобы разрешить доступ клиенту, где-то глубоко внутри скрываются те самые разрешения, которые запрашивает клиент. Эти разрешения, называемые **областью действия** (scope), являются еще одной важной функцией OAuth 2. Они позволяют клиенту запрашивать ограниченный доступ к данным пользователя, тем самым облегчая пользователю доверие к клиенту.

Что делает область действия действительно мощным инструментом, так это то, что её ограничения связаны с конкретным клиентом. В отличие от ключа API, где ограничения, налагаемые на ключ, одинаково влияют на каждого клиента, область действия OAuth позволяет одному клиенту иметь разрешение X, а другому — разрешения X и Y. Это означает, что один веб-сайт может просматривать ваши контакты, а другой сайт может просматривать и редактировать их.